Deep_Learning_Project_Notebook

March 2, 2026

1 Deep Learning : Project 1 : Predicting Song Genre from Album Art Cover

1.0.1 Justin Stutler : 1/30/2026 - 3/1/2026

Note: Numbers in parenthesis indicate the associated rubric section and may be out of order due to coding structure. ____ # (1.) Project Goal

This project performed an ablation study on residual connections within convolutional neural networks for the task of predicting song genre from album cover art. The study compared 4 custom architectures across a 2×2 grid of depth (shallow vs deep) and connection type (CNN vs ResNet), with each architecture tested under 4 training conditions (baseline, batch normalization, data augmentation, dropout) to isolate the effect of individual design choices. CLIP (ViT-B/32) was included as a foundation model upper bound.

The ablation study was a success. Some hypotheses were confirmed; ResNet-18 became the best custom model with batch normalization and CLIP dominated by 2x+ on every metric. Others held contradictory results; without batch normalization the CNNs actually outperformed the ResNets, and deeper models performed worse than shallow ones at baseline due to gradient instability. The study also produced insights that were not hypothesized, such as data augmentation causing mode collapse on deep networks without normalization and dropout being counterproductive on underfitting models.

Classification performance on this task was low, with custom models reaching 15-20% top-1 accuracy and CLIP reaching 44%. This reflects a weak visual signal between album cover art and genre rather than a flaw in the methodology. The dataset has only 1,000 images per genre and album art is not algorithmically tied to genre, so a more diverse and larger dataset would likely improve raw accuracy. The study's value is in the architectural comparisons and technique interactions, not in maximizing accuracy on this particular task.

Note: The proposal originally described CLIP as zero-shot only. I changed to linear probing during implementation because training a classification head is more consistent with the methodology of training all models as classifiers on the same labeled data.

2 (2.) Input and Ground Truth

The dataset contains images of size 300 x 300. The ResNet-18 architecture typically receives inputs of size 224 x 224, so the images are resized during preprocessing. Each image contains RGB values for each pixel in the album cover art image. The ground truth is the actual genre of the given image. This dataset contains 20 distinct genres stored as folder names (strings) which are mapped

to integer class indices by PyTorch's ImageFolder. The album cover art images are used to make a classification prediction of which genre the image belongs to. The ground truth is used to evaluate the accuracy of the prediction. No changes were made to the input representation or ground truth labels from the proposal stage. The images and genre labels are used exactly as described in the original plan.

2.1 Setting up the Environment

2.1.1 Install Necessary Packages

Ensure the cells are uncommented (remove #) if you wish to train the models yourself

- **torch** — building and training neural networks
- **torchvision** — image datasets and transforms
- **torchaudio** — included with the PyTorch bundle

[1]:
```
#%pip install torch torchvision torchaudio --upgrade --index-url https://
↪download.pytorch.org/whl/cu118
```

- **scikit-learn** — evaluation metrics (precision, recall, f1, confusion matrix)
- **seaborn** — heatmaps and bar plots
- **numpy** — array operations
- **ipywidgets / tqdm** — progress bars
- **matplotlib** — plotting
- **kaggle** — dataset download
- **IPython** — display utilities
- **transformers** — CLIP foundation model

[2]:
```
#%pip install scikit-learn seaborn "numpy<2" ipywidgets tqdm matplotlib -q␣
↪kaggle IPython transformers
```

2.1.2 Imports

[3]:
```
import torch
import torch.nn as nn
import torch.optim as optim
import numpy as np
import matplotlib.pyplot as plt
import os
import shutil
import json
import warnings
import seaborn as sns
import pandas as pd
from torchvision import datasets, transforms
from torch.utils.data import DataLoader, Subset
from sklearn.model_selection import train_test_split
```

```
from sklearn.metrics import confusion_matrix, precision_score, recall_score,␣
↪f1_score
from tqdm.notebook import tqdm
from itertools import islice
from IPython.display import display, HTML
```

2.1.3 Use GPU with CUDA

[4]:
```
print("GPU Available: ", torch.cuda.is_available())
if torch.cuda.is_available():
    print("GPU Name: ", torch.cuda.get_device_name(0))

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Using device: {device}")
```

```
GPU Available:  False
Using device: cpu
```

2.1.4 Configuration

I trained the models without specifying a seed. However, I have added a seed for future improvements.
All of the model weights and metrics from my run are saved and stored in the project file. These weights and metrics are loaded if found, else the models will begin training.

[5]:
```
# set seeds for reproducibility across pytorch, numpy, and cuda
SEED = 42
torch.manual_seed(SEED)
np.random.seed(SEED)
if torch.cuda.is_available():
    torch.cuda.manual_seed_all(SEED)
torch.backends.cudnn.deterministic = True
torch.backends.cudnn.benchmark = False   # trades speed for exact␣
↪reproducibility

# same hyperparameters used across all experiments
LEARNING_RATE = 0.001
EPOCHS = 21
PATIENCE = 5          # early stopping patience (applied to every run)
OPTIMIZER_NAME = "Adam"
LOSS_FN_NAME = "CrossEntropyLoss"
plot_size = (14, 10)

print("--- Reproducibility & Environment Info ---")
print(f"PyTorch version : {torch.__version__}")
print(f"CUDA version    : {torch.version.cuda}")
if torch.cuda.is_available():
    print(f"GPU             : {torch.cuda.get_device_name(0)}")
```

```
print(f"Random seed     : {SEED}")
print(f"Optimizer       : {OPTIMIZER_NAME}  |  LR: {LEARNING_RATE}")
print(f"Loss function   : {LOSS_FN_NAME}")
print(f"Max epochs      : {EPOCHS}  |  Early-stop patience: {PATIENCE}")
```

```
--- Reproducibility & Environment Info ---
PyTorch version : 2.7.1+cu118
CUDA version    : 11.8
Random seed     : 42
Optimizer       : Adam  |  LR: 0.001
Loss function   : CrossEntropyLoss
Max epochs      : 21  |  Early-stop patience: 5
```

2.2 Preparing the Dataset

---

3 (3.) Dataset Description

This project uses a public dataset called '20k Album Covers within 20 Genres' from Kaggle: https://www.kaggle.com/datasets/michaeljkerr/20k-album-covers-within-20-genres/data

The dataset contains 20,000 images across 20 genres with exactly 1,000 images per genre, so the class distribution is perfectly balanced. Images are 300×300 pixels stored as JPG files. Each genre is stored in its own subfolder.

3.0.1 Import Data from Kaggle

(Ensure Kaggle.json is in directory to enable download)

[6]:
```
kaggle_dir = os.path.expanduser('~/.kaggle')
kaggle_json_src = "kaggle.json"
kaggle_json_dest = os.path.join(kaggle_dir, "kaggle.json")

if not os.path.exists(kaggle_dir):
    os.makedirs(kaggle_dir)

if os.path.exists(kaggle_json_src):
    shutil.move(kaggle_json_src, kaggle_json_dest)
    print(f"Kaggle credentials moved to: {kaggle_json_dest}")
elif os.path.exists(kaggle_json_dest):
    print("Kaggle credentials already in place.")
else:
    print("WARNING: kaggle.json not found. Please ensure it is uploaded to this␣
 ↪directory.")

# dataset name and location
```

```
dataset_name = "michaeljkerr/20k-album-covers-within-20-genres"
download_path = "./data"

# download data if dataset folder does not exist
if not os.path.exists(download_path):
    os.makedirs(download_path)
    print("Downloading dataset...")
    import kaggle
    kaggle.api.authenticate()
    kaggle.api.dataset_download_cli(dataset_name, path=download_path,␣
 ↪unzip=True)
    print("Dataset ready in ./data")
else:
    print("Dataset folder already exists.")
```

```
Kaggle credentials already in place.
Dataset folder already exists.
```

4 (4.) Data Processing

Images are resized from 300×300 to 224×224 to match the input size expected by the ResNet-style architectures. Each image is converted to a tensor and normalized using ImageNet mean ([0.485, 0.456, 0.406]) and standard deviation ([0.229, 0.224, 0.225]) values. I use ImageNet normalization because the pixel distributions of album cover art are similar enough to natural images that these statistics keep the activations in a stable range during training.

The dataset is split into training (80%), validation (10%), and test (10%) using stratified sampling with a fixed random state of 42. Stratified splitting ensures each genre has proportional representation across all three sets so no genre is over or underrepresented in any split. The split is performed on the indices of the full dataset using sklearn's train_test_split, then PyTorch Subset objects are created from those indices.

Data leakage is avoided because the split is done before any training begins, and all three subsets utilize the same transform pipeline. No information from the validation or test sets is used during training. When data augmentation is applied (technique 2), augmented transforms are only applied to the training subset; validation and test sets always use the standard resize-and-normalize pipeline so evaluation is done on clean unaugmented images.

For CLIP, a separate transform pipeline is used with CLIP-specific normalization values ([0.48145, 0.4578, 0.4082] mean, [0.2686, 0.2613, 0.2758] std) since CLIP was pretrained with different image statistics than ImageNet. The same train/val/test indices are reused so all models are evaluated on identical data splits.

4.0.1 Exploring the Data

[7]:
```
#  specify where data is stored
data_dir = './data/GAID'
BATCH_SIZE = 32
NUM_WORKERS = 0
```

```
data_transforms = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
])

try:
    full_dataset = datasets.ImageFolder(data_dir, transform=data_transforms)
    print(f"Classes found: {len(full_dataset.classes)}")

    # stratified split: 80% train, 10% val, 10% test
    targets = full_dataset.targets
    indices = np.arange(len(targets))

    train_idx, temp_idx, train_targets, temp_targets = train_test_split(
        indices, targets, test_size=0.20, stratify=targets, random_state=42
    )

    val_idx, test_idx, _, _ = train_test_split(
        temp_idx, temp_targets, test_size=0.50, stratify=temp_targets,␣
 ↪random_state=42
    )

    train_dataset = Subset(full_dataset, train_idx)
    val_dataset = Subset(full_dataset, val_idx)
    test_dataset = Subset(full_dataset, test_idx)

    print(f"Stratified Split Successful:")
    print(f"Train: {len(train_dataset)} | Val: {len(val_dataset)} | Test:␣
 ↪{len(test_dataset)}")

    train_loader = DataLoader(train_dataset, batch_size=BATCH_SIZE,␣
 ↪shuffle=True,
                              num_workers=NUM_WORKERS, pin_memory=True)
    val_loader = DataLoader(val_dataset, batch_size=BATCH_SIZE, shuffle=False,
                            num_workers=NUM_WORKERS, pin_memory=True)
    test_loader = DataLoader(test_dataset, batch_size=BATCH_SIZE, shuffle=False,
                             num_workers=NUM_WORKERS, pin_memory=True)
    print("DataLoaders created successfully.")

    def imshow(img, title=None):
        """Display a single image tensor after un-normalizing."""
        img = img.numpy().transpose((1, 2, 0))
        mean = np.array([0.485, 0.456, 0.406])
        std = np.array([0.229, 0.224, 0.225])
        img = std * img + mean
```

```
        img = np.clip(img, 0, 1)
        plt.figure(figsize=plot_size)
        plt.imshow(img)
        if title: plt.title(title)
        plt.axis('off')
        plt.show()

    images, labels = next(iter(train_loader))
    print(f"\nExample image (genre: {full_dataset.classes[labels[0]]})")
    imshow(images[0])

except Exception as e:
    print(f"Error loading data: {e}")
```

```
Classes found: 20
Stratified Split Successful:
Train: 16000 | Val: 2000 | Test: 2000
DataLoaders created successfully.

Example image (genre: Techno)
```

```
C:\Users\justi\anaconda3\Lib\site-packages\torch\utils\data\dataloader.py:665:
UserWarning: 'pin_memory' argument is set as true but no accelerator is found,
then device pinned memory won't be used.
  warnings.warn(warn_msg)
```

---

5 (5.) Methodology

This project is structured as an ablation study. I compare 4 custom architectures across 4 training configurations to isolate the effect of specific design choices. The two variables being tested are depth (shallow vs deep) and residual connections (CNN vs ResNet), which gives a 2×2 grid of architectures:

	Shallow	Deep
CNN	cnn_10 (3 conv blocks)	cnn_18 (5 conv blocks)
ResNet	resnet_10 (2 residual layers)	resnet_18 (4 residual layers)

Each architecture is trained as a baseline and then 3 techniques will be applied individually so the impact of each technique can be compared(see below).

1. **Baseline** — no techniques applied, establishes baseline performance
2. **Batch Normalization** — normalizes activations after each convolution to stabilize gradient flow
3. **Data Augmentation** — random crops, flips, and rotations applied to training images only
4. **Dropout** — 50% of classifier neurons randomly zeroed during training to reduce overfitting

This means 16 total training runs (4 architectures × 4 conditions) for the custom models. After this, the foundation model, CLIP, will be trained using a linear classification head and evaluated.

The training order follows a logical progression. Baseline reveals each model's raw strengths and weaknesses. Batch normalization targets gradient instability which is the deep models' main problem. Data augmentation targets overfitting by increasing effective dataset diversity. Dropout targets overfitting from a different angle by forcing redundant feature learning. CLIP provides an upper bound showing what pretrained features can achieve on the same task.

5.1 (5.2) Define Model Architectures

(Architectures are defined here before training begins in Section 5.1 below) _____

This project compares 4 custom architectures: - **cnn_10** — shallow CNN, 3 convolutional blocks - **cnn_18** — deep CNN, 5 convolutional blocks - **resnet_10** — shallow ResNet, 2 residual layers - **resnet_18** — deep ResNet, 4 residual layers

All 4 share the same classifier head and adaptive pooling; the difference is in how the feature extraction layers are built. A foundation model (CLIP) is also evaluated as a comparison.

CNN Architectures (cnn_10, cnn_18) Both CNNs stack ConvolutionalBlocks where each block follows the same pattern: a 3×3 convolution (stride 1, padding 1) followed by ReLU activation and 2×2 max pooling. The shallow CNN (cnn_10) uses 3 blocks with channel progression 3→32→64→128. The deep CNN (cnn_18) uses 5 blocks expanding to 3→32→64→128→256→512. More blocks means the network can learn increasingly abstract features, but it also means more parameters and a higher risk of overfitting on a small dataset.

ResNet Architectures (resnet_10, resnet_18) Both ResNets use an initial 7×7 convolution (stride 2) followed by 3×3 max pooling to downsample the input early. After that, they stack ResidualBlocks where each block contains two 3×3 convolutions with a skip connection that adds the input directly to the output. This skip connection is the key difference from the CNN; it gives the gradient a shortcut path during backpropagation so deeper layers can still receive useful gradient signal. The shallow ResNet (resnet_10) has 2 residual layers with channels 64→128. The deep ResNet (resnet_18) has 4 residual layers expanding 64→128→256→512, matching the standard ResNet-18 structure. When the channel dimensions change between layers, a 1×1 convolution downsamples the skip connection to match.

CLIP Architecture (ViT-B/32) CLIP uses a Vision Transformer (ViT-B/32) as its backbone instead of convolutional layers. The image is split into 32×32 pixel patches which are flattened

and projected into embeddings, then processed through 12 transformer encoder layers with self-attention. This allows the model to learn relationships between all patches in the image simultaneously rather than building up features through local receptive fields like a CNN. The backbone was pretrained by OpenAI on roughly 400 million image-text pairs to align visual and text representations in a shared embedding space. For this project I freeze the entire backbone and attach a single linear classification layer on top of the pooled output. Only this classification head is trained; the backbone weights do not change. This is linear probing — I am testing whether CLIP's pretrained features are useful for our task, not fine-tuning CLIP itself.

Shared Components All 4 custom models end with adaptive average pooling that collapses spatial dimensions to 1×1, then a fully connected linear layer that maps to 20 output nodes (one per genre). CLIP's classifier head follows the same pattern with a linear layer mapping from CLIP's 768-dimensional pooled output to 20 genre classes. During training, CrossEntropyLoss applies softmax internally to convert these raw logits into a probability distribution over the 20 genres. The predicted genre is whichever class has the highest logit value. This means the models output raw scores and the loss function handles the softmax; there is no explicit softmax layer in any of the architectures.

Batch normalization and dropout are toggled on/off via flags in the custom models so the same architecture can be reused across technique experiments without rewriting the model. When batch normalization is enabled, a BatchNorm2d layer is inserted after every convolution. When dropout is enabled, a Dropout(p=0.5) layer is placed before the final linear classifier. The CLIP classifier head uses Dropout(p=0.5) by default since it is only training a single layer and benefits from regularization.

Why These Architectures I designed the comparison this way to isolate two variables: depth (shallow vs deep) and residual connections (CNN vs ResNet). By crossing these two variables I can evaluate whether skip connections actually help, and whether the benefit scales with depth. The expectation is that deeper models learn richer features but suffer from vanishing gradients, and that residual connections should fix this especially in the 18-layer variants. CLIP serves as an upper bound showing what a pretrained model with massively more training data can achieve on the same task.

[8]:
```
class ConvolutionalBlock(nn.Module):
    """Conv2d -> optional BatchNorm -> ReLU -> MaxPool."""
    def __init__(self, in_channels, out_channels, use_batch_norm=False):
        super().__init__()
        layers = [nn.Conv2d(in_channels, out_channels, kernel_size=3,␣
 ↪padding=1, stride=1)]
        if use_batch_norm: layers.append(nn.BatchNorm2d(out_channels))
        layers.extend([nn.ReLU(), nn.MaxPool2d(kernel_size=2, stride=2)])
        self.block = nn.Sequential(*layers)

    def forward(self, x): return self.block(x)

class ResidualBlock(nn.Module):
    """Two-conv residual block with optional batch normalization and␣
 ↪downsampling."""
```

```
    def __init__(self, in_channels, out_channels, use_batch_norm=False,␣
↪stride=1, downsample=None):
        super().__init__()
        self.use_batch_norm = use_batch_norm
        self.conv1 = nn.Conv2d(in_channels, out_channels, kernel_size=3,␣
↪stride=stride, padding=1)
        if self.use_batch_norm: self.bn1 = nn.BatchNorm2d(out_channels)
        self.relu = nn.ReLU()
        self.conv2 = nn.Conv2d(out_channels, out_channels, kernel_size=3,␣
↪stride=1, padding=1)
        if self.use_batch_norm: self.bn2 = nn.BatchNorm2d(out_channels)
        self.downsample = downsample

    def forward(self, x):
        identity = x
        out = self.conv1(x)
        if self.use_batch_norm: out = self.bn1(out)
        out = self.relu(out)
        out = self.conv2(out)
        if self.use_batch_norm: out = self.bn2(out)
        if self.downsample is not None: identity = self.downsample(x)
        out += identity
        return self.relu(out)

class BaseCNN(nn.Module):
    """Base class for all CNN models with shared pooling and classifier logic.
↪"""
    def __init__(self, use_batch_norm=False, use_dropout=False, num_classes=20):
        super().__init__()
        self.use_batch_norm = use_batch_norm
        self.use_dropout = use_dropout
        self.num_classes = num_classes
        self.adaptive_pool = nn.AdaptiveAvgPool2d((1, 1))

    def _build_classifier(self, in_features):
        layers = [nn.Dropout(p=0.5)] if self.use_dropout else []
        layers.append(nn.Linear(in_features, self.num_classes))
        return nn.Sequential(*layers)

class CNNSmall(BaseCNN):
    """3-block convolutional network."""
    def __init__(self, use_batch_norm=False, use_dropout=False, num_classes=20):
        super().__init__(use_batch_norm, use_dropout, num_classes)
        self.features = nn.Sequential(
            ConvolutionalBlock(3, 32, self.use_batch_norm),
            ConvolutionalBlock(32, 64, self.use_batch_norm),
            ConvolutionalBlock(64, 128, self.use_batch_norm)
```

```
        )
        self.classifier = self._build_classifier(in_features=128)

    def forward(self, x):
        x = self.features(x)
        x = self.adaptive_pool(x)
        x = torch.flatten(x, 1)
        return self.classifier(x)

class ResNetSmall(BaseCNN):
    """Shallow ResNet with 2 residual layers."""
    def __init__(self, use_batch_norm=False, use_dropout=False, num_classes=20):
        super().__init__(use_batch_norm, use_dropout, num_classes)
        self.in_channels = 64
        self.conv1 = nn.Conv2d(3, self.in_channels, kernel_size=7, stride=2,␣
 ↪padding=3)
        self.bn1 = nn.BatchNorm2d(self.in_channels) if self.use_batch_norm else␣
 ↪nn.Identity()
        self.relu = nn.ReLU()
        self.maxpool = nn.MaxPool2d(kernel_size=3, stride=2, padding=1)
        self.layer1 = self._make_layer(out_channels=64, blocks=2, stride=1)
        self.layer2 = self._make_layer(out_channels=128, blocks=2, stride=2)
        self.classifier = self._build_classifier(in_features=128)

    def _make_layer(self, out_channels, blocks, stride):
        downsample = None
        if stride != 1 or self.in_channels != out_channels:
            down_layers = [nn.Conv2d(self.in_channels, out_channels,␣
 ↪kernel_size=1, stride=stride)]
            if self.use_batch_norm: down_layers.append(nn.
 ↪BatchNorm2d(out_channels))
            downsample = nn.Sequential(*down_layers)
        layers = [ResidualBlock(self.in_channels, out_channels, self.
 ↪use_batch_norm, stride, downsample)]
        self.in_channels = out_channels
        for _ in range(1, blocks):
            layers.append(ResidualBlock(self.in_channels, out_channels, self.
 ↪use_batch_norm))
        return nn.Sequential(*layers)

    def forward(self, x):
        x = self.maxpool(self.relu(self.bn1(self.conv1(x))))
        x = self.layer2(self.layer1(x))
        x = torch.flatten(self.adaptive_pool(x), 1)
        return self.classifier(x)
```

```python
class CNNLarge(BaseCNN):
    """5-block convolutional network."""
    def __init__(self, use_batch_norm=False, use_dropout=False, num_classes=20):
        super().__init__(use_batch_norm, use_dropout, num_classes)
        self.features = nn.Sequential(
            ConvolutionalBlock(3, 32, self.use_batch_norm),
            ConvolutionalBlock(32, 64, self.use_batch_norm),
            ConvolutionalBlock(64, 128, self.use_batch_norm),
            ConvolutionalBlock(128, 256, self.use_batch_norm),
            ConvolutionalBlock(256, 512, self.use_batch_norm)
        )
        self.classifier = self._build_classifier(in_features=512)

    def forward(self, x):
        x = self.features(x)
        x = self.adaptive_pool(x)
        x = torch.flatten(x, 1)
        return self.classifier(x)


class ResNetLarge(BaseCNN):
    """Deep ResNet with 4 residual layers (similar to ResNet-18)."""
    def __init__(self, use_batch_norm=False, use_dropout=False, num_classes=20):
        super().__init__(use_batch_norm, use_dropout, num_classes)
        self.in_channels = 64
        self.conv1 = nn.Conv2d(3, self.in_channels, kernel_size=7, stride=2,␣
↪padding=3)
        self.bn1 = nn.BatchNorm2d(self.in_channels) if self.use_batch_norm else␣
↪nn.Identity()
        self.relu = nn.ReLU()
        self.maxpool = nn.MaxPool2d(kernel_size=3, stride=2, padding=1)
        self.layer1 = self._make_layer(out_channels=64, blocks=2, stride=1)
        self.layer2 = self._make_layer(out_channels=128, blocks=2, stride=2)
        self.layer3 = self._make_layer(out_channels=256, blocks=2, stride=2)
        self.layer4 = self._make_layer(out_channels=512, blocks=2, stride=2)
        self.classifier = self._build_classifier(in_features=512)

    def _make_layer(self, out_channels, blocks, stride):
        downsample = None
        if stride != 1 or self.in_channels != out_channels:
            down_layers = [nn.Conv2d(self.in_channels, out_channels,␣
↪kernel_size=1, stride=stride)]
            if self.use_batch_norm: down_layers.append(nn.
↪BatchNorm2d(out_channels))
            downsample = nn.Sequential(*down_layers)
        layers = [ResidualBlock(self.in_channels, out_channels, self.
↪use_batch_norm, stride, downsample)]
```

```
        self.in_channels = out_channels
        for _ in range(1, blocks):
            layers.append(ResidualBlock(self.in_channels, out_channels, self.
↪use_batch_norm))
        return nn.Sequential(*layers)

    def forward(self, x):
        x = self.maxpool(self.relu(self.bn1(self.conv1(x))))
        x = self.layer1(x)
        x = self.layer2(x)
        x = self.layer3(x)
        x = self.layer4(x)
        x = torch.flatten(self.adaptive_pool(x), 1)
        return self.classifier(x)
```

5.2 (5.3) Training Details and Reproducibility

All models are trained using the same hyperparameters so differences in performance can be attributed to architecture and technique rather than training configuration.

- **Framework:** PyTorch (with torchvision for data loading and transforms)
- **Optimizer:** Adam with learning rate 0.001
- **Loss Function:** CrossEntropyLoss (standard for multi-class classification)
- **Batch Size:** 32
- **Max Epochs:** 21
- **Early Stopping:** patience of 5 epochs on validation loss (applied to every run including baseline)
- **Random Seed:** 42 (set for PyTorch, NumPy, and CUDA)
- **Hardware:** NVIDIA GPU via CUDA

I trained the models without specifying a seed initially. The seed and deterministic flags were added afterward for reproducibility of future runs. All model weights and metrics from my original training run are saved to disk and loaded automatically if found, so results are consistent across notebook restarts.

5.2.1 Train Model Function

[9]:
```
def train_model(model, train_loader, val_loader, criterion, optimizer,␣
↪epochs=21, device='cpu', debug=False, patience=5):
    """Train the model with early stopping. If debug=True, only runs 3 epochs␣
↪on 1 batch each."""
    history = {'train_loss': [], 'val_loss': [], 'val_acc': [], 'val_top5_acc':␣
↪[]}

    debug_limit = 1
    best_val_loss = float('inf')
    epochs_no_improve = 0
```

```
    if debug:
        print(f"DEBUG MODE ACTIVE: Running 3 epochs, {debug_limit} batch(es)␣
↪max.")
        epochs = 3

    for epoch in tqdm(range(epochs), desc="Total Training Time", unit="epoch"):
        # training phase
        model.train()
        running_loss = 0.0
        train_samples = 0

        current_train_loader = islice(train_loader, debug_limit) if debug else␣
↪train_loader
        train_total = debug_limit if debug else len(train_loader)
        train_bar = tqdm(current_train_loader, total=train_total, desc=f'Epoch␣
↪{epoch+1}/{epochs} [Train]', leave=False)

        for inputs, labels in train_bar:
            inputs, labels = inputs.to(device), labels.to(device)
            optimizer.zero_grad()
            outputs = model(inputs)
            loss = criterion(outputs, labels)
            loss.backward()
            optimizer.step()
            running_loss += loss.item() * inputs.size(0)
            train_samples += inputs.size(0)
            train_bar.set_postfix(loss=loss.item())

        epoch_train_loss = running_loss / train_samples
        history['train_loss'].append(epoch_train_loss)

        # validation phase
        model.eval()
        val_loss = 0.0
        correct = 0
        correct_top5 = 0
        val_samples = 0

        current_val_loader = islice(val_loader, debug_limit) if debug else␣
↪val_loader
        val_total = debug_limit if debug else len(val_loader)
        val_bar = tqdm(current_val_loader, total=val_total, desc=f'Epoch␣
↪{epoch+1}/{epochs} [Val]', leave=False)

        with torch.no_grad():
            for inputs, labels in val_bar:
                inputs, labels = inputs.to(device), labels.to(device)
```

```
                outputs = model(inputs)
                loss = criterion(outputs, labels)
                val_loss += loss.item() * inputs.size(0)
                _, preds = torch.max(outputs, 1)
                correct += torch.sum(preds == labels.data)
                _, top5_preds = torch.topk(outputs, k=5, dim=1)
                correct_top5 += torch.sum(top5_preds == labels.view(-1, 1)).
↪item()
                val_samples += inputs.size(0)

        epoch_val_loss = val_loss / val_samples
        epoch_val_acc = correct.double() / val_samples
        epoch_val_top5_acc = correct_top5 / val_samples

        history['val_loss'].append(epoch_val_loss)
        history['val_acc'].append(epoch_val_acc.item())
        history['val_top5_acc'].append(epoch_val_top5_acc)

        print(f"Epoch {epoch+1}/{epochs} | Train Loss: {epoch_train_loss:.4f} |␣
↪Val Loss: {epoch_val_loss:.4f} | Val Acc: {epoch_val_acc:.4f} | Val Top-5␣
↪Acc: {epoch_val_top5_acc:.4f}")

        # early stopping
        if epoch_val_loss < best_val_loss:
            print(f"Validation loss decreased ({best_val_loss:.4f} -->␣
↪{epoch_val_loss:.4f}). Saving best weights.")
            best_val_loss = epoch_val_loss
            torch.save(model.state_dict(), 'best_model_weights.pth')
            print(f"New best model saved at epoch {epoch+1}.")
            epochs_no_improve = 0
        else:
            epochs_no_improve += 1
            if epochs_no_improve >= patience:
                print(f"Early stopping triggered after {epoch+1} epochs.")
                break

    print(f"\nTraining complete. Restoring best model weights (Best Val Loss:␣
↪{best_val_loss:.4f})")
    model.load_state_dict(torch.load('best_model_weights.pth'))
    return history
```

5.2.2 Load Weights

[10]:
```
def load_weights(model, weights_path, device='cpu'):
    """Load model weights and training history from a checkpoint file."""
    if not weights_path.startswith('model_weights'):
        weights_path = os.path.join('model_weights', weights_path)
```

```
    if not os.path.exists(weights_path):
        return None

    ckpt = torch.load(weights_path, map_location=device, weights_only=False)
    state = ckpt["state_dict"] if isinstance(ckpt, dict) and "state_dict" in␣
↪ckpt else ckpt

    try:
        model.load_state_dict(state, strict=True)
    except RuntimeError:
        print(f"Checkpoint architecture mismatch: {weights_path}")
        return None

    history = ckpt.get("history", {}) if isinstance(ckpt, dict) else {}
    print(f"Loaded weights from {weights_path}")
    return history
```

5.2.3 Save Weights

[11]:
```
def save_weights(model, weights_path, history=None):
    """Save model weights and training history to a checkpoint file."""
    if not weights_path.startswith('model_weights'):
        weights_path = os.path.join('model_weights', weights_path)
    os.makedirs(os.path.dirname(weights_path), exist_ok=True)
    torch.save({
        "state_dict": model.state_dict(),
        "history": history or {},
        "model_class": type(model).__name__
    }, weights_path)
    print(f"Saved weights to {weights_path}")
```

5.2.4 Store Model Metrics

(precision, recall, f-score, top-1 accuracy, top-5 accuracy, top n confused pairs)

[12]:
```
class ModelMetricsStore:
    """Centralized read/write for per-model metrics in model_metrics.json."""

    def __init__(self, filepath="model_metrics.json"):
        self.filepath = filepath

    def _read(self):
        if os.path.exists(self.filepath):
            with open(self.filepath, "r") as f:
                try:
                    return json.load(f)
                except json.JSONDecodeError:
```

```
                    return {}
        return {}

    def _write(self, data):
        with open(self.filepath, "w") as f:
            json.dump(data, f, indent=4)

    def load(self, model_name):
        """Return metrics dict for model_name, or None if missing/incomplete."""
        REQUIRED = {"precision", "recall", "f1", "top1_acc", "top5_acc",␣
↪"top_confused_pairs"}
        data = self._read()
        if model_name in data and REQUIRED.issubset(data[model_name].keys()):
            return data[model_name]
        return None

    def save(self, model_name, metrics_dict):
        """Save metrics dict under model_name key."""
        data = self._read()
        data[model_name] = metrics_dict
        self._write(data)

    def load_all(self):
        """Return entire metrics dictionary."""
        return self._read()

metrics_store = ModelMetricsStore()
```

5.2.5 Get Predictions

[13]:
```
def get_predictions(model, data_loader, device='cpu', debug=False):
    """Run model on data_loader and return (y_true, y_pred, logits)."""
    model.eval()
    y_true_list, y_pred_list, logits_list = [], [], []
    loader = islice(data_loader, 1) if debug else data_loader
    with torch.no_grad():
        for inputs, labels in loader:
            inputs = inputs.to(device)
            outputs = model(inputs)
            _, preds = torch.max(outputs, 1)
            y_true_list.append(labels.numpy())
            y_pred_list.append(preds.cpu().numpy())
            logits_list.append(outputs.cpu().numpy())
    return (np.concatenate(y_true_list),
            np.concatenate(y_pred_list),
            np.concatenate(logits_list))
```

5.2.6 Compute Top k Accuracy

used for top 1 and top 5

[14]:
```
def compute_top_k_accuracy(y_true, logits, k=5):
    """Compute top-k accuracy from true labels and raw logits (numpy arrays)."""
    y_true = np.asarray(y_true).ravel()
    top_k_preds = np.argsort(logits, axis=1)[:, -k:]
    correct = np.any(top_k_preds == y_true[:, None], axis=1)
    return float(correct.mean())
```

5.2.7 Get Top N Confused Pairs

[15]:
```
def get_top_confused_pairs(cm, class_names, top_n=1):
    """Return the top_n most-confused (true, predicted) class pairs from a␣
 ↪confusion matrix."""
    pairs = []
    n = len(class_names)
    for i in range(n):
        for j in range(n):
            if i != j and cm[i, j] > 0:
                pairs.append({"true": class_names[i], "predicted":␣
 ↪class_names[j], "count": int(cm[i, j])})
    pairs.sort(key=lambda x: x["count"], reverse=True)
    return pairs[:top_n]
```

5.2.8 Load/Compute Metrics

Load Metrics if exist, else Compute and save

[16]:
```
# specify top n confused paits per model to evaluate
TOP_N_CONFUSED = 3 # default 3

def load_or_compute_metrics(metrics_key, model, data_loader, class_names,␣
 ↪device, top_n=None):
    """Load metrics from cache or compute fresh from predictions.

    Returns (precision, recall, f1, top1_acc, top5_acc, top_confused_pairs,␣
 ↪y_true, y_pred).
    y_true/y_pred are None when loaded from cache.
    """
    if top_n is None:
        top_n = TOP_N_CONFUSED

    cached = metrics_store.load(metrics_key)
    if cached is not None:
        print(f"Loaded existing metrics for {metrics_key}")
        return (cached["precision"], cached["recall"], cached["f1"],
```

```
                cached["top1_acc"], cached["top5_acc"],␣
↪cached["top_confused_pairs"],
                None, None)

    # compute fresh
    y_true, y_pred, logits = get_predictions(model, data_loader, device=device)

    y_true_int = np.asarray(y_true).ravel().astype(int)
    y_pred_int = np.asarray(y_pred).ravel().astype(int)
    with warnings.catch_warnings():
        warnings.filterwarnings("ignore", category=UserWarning)
        precision = precision_score(y_true_int, y_pred_int, average="weighted",␣
↪zero_division=0)
        recall = recall_score(y_true_int, y_pred_int, average="weighted",␣
↪zero_division=0)
        f1 = f1_score(y_true_int, y_pred_int, average="weighted",␣
↪zero_division=0)

    top1_acc = float((y_true_int == y_pred_int).mean())
    top5_acc = compute_top_k_accuracy(y_true, logits, k=5)
    cm = confusion_matrix(y_true, y_pred, labels=np.arange(len(class_names)))
    top_confused = get_top_confused_pairs(cm, class_names, top_n=top_n)

    metrics_store.save(metrics_key, {
        "precision": precision, "recall": recall, "f1": f1,
        "top1_acc": top1_acc, "top5_acc": top5_acc, "top_confused_pairs":␣
↪top_confused
    })

    return precision, recall, f1, top1_acc, top5_acc, top_confused, y_true,␣
↪y_pred
```

5.2.9 Loss Curve Comparison Plot

[17]:
```
def plot_loss_comparison(histories, model_names, title="Loss Comparison"):
    """Plot training and validation loss curves for multiple models on one␣
↪graph.
    histories: dict mapping model_name -> history dict with 'train_loss' and␣
↪'val_loss' keys.
    """
    colors = plt.rcParams['axes.prop_cycle'].by_key()['color']
    plt.figure(figsize=plot_size)

    for i, name in enumerate(model_names):
        data = histories.get(name, {})
        t_loss = data.get('train_loss', [])
```

```
        v_loss = data.get('val_loss', [])
        if not t_loss or not v_loss:
            print(f"Skipping '{name}': No loss data found.")
            continue
        epochs = range(1, len(t_loss) + 1)
        c = colors[i % len(colors)]
        plt.plot(epochs, t_loss, linestyle='-', marker='o', color=c,␣
↪label=f'{name} (Train)')
        plt.plot(epochs, v_loss, linestyle='--', marker='s', color=c,␣
↪label=f'{name} (Val)')

    plt.title(title)
    plt.xlabel("Epochs")
    plt.ylabel("Loss")
    plt.grid(True)
    plt.legend(bbox_to_anchor=(1.05, 1), loc='upper left')
    plt.tight_layout()
    plt.show()
```

5.2.10 Accuracy Comparison Plot

[18]:
```
def plot_accuracy_comparison(histories, model_names, acc_key='val_acc',␣
↪ylabel='Top-1 Accuracy', title="Accuracy Comparison"):
    """Plot validation accuracy curves for multiple models on one graph."""
    colors = plt.rcParams['axes.prop_cycle'].by_key()['color']
    plt.figure(figsize=plot_size)

    for i, name in enumerate(model_names):
        data = histories.get(name, {})
        acc = data.get(acc_key, [])
        if not acc:
            print(f"Skipping '{name}': No {acc_key} data found.")
            continue
        epochs = range(1, len(acc) + 1)
        c = colors[i % len(colors)]
        plt.plot(epochs, acc, linestyle='--', marker='s', color=c,␣
↪label=f'{name}')

    plt.title(title)
    plt.xlabel("Epochs")
    plt.ylabel(ylabel)
    plt.grid(True)
    plt.legend(bbox_to_anchor=(1.05, 1), loc='upper left')
    plt.tight_layout()
    plt.show()
```

5.2.11 Confusion Matrix Plot

[19]:
```
def plot_confusion_matrix(y_true, y_pred, class_names, model_name="Model"):
    """Plot a confusion matrix heatmap."""
    cm = confusion_matrix(y_true, y_pred, labels=np.arange(len(class_names)))
    plt.figure(figsize=plot_size)
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
                xticklabels=class_names, yticklabels=class_names)
    plt.title(f'{model_name}: Confusion Matrix')
    plt.xlabel('Predicted Genre')
    plt.ylabel('Actual Genre')
    plt.xticks(rotation=45, ha='right')
    plt.tight_layout()
    plt.show()
```

5.2.12 F-Score Plot

[20]:
```
def plot_f_score(metric_scores, phase_label):
    """Plot precision, recall, and F1 bar charts side-by-side."""
    fig, axes = plt.subplots(1, 3, figsize=plot_size, sharey=True)

    metric_plots = [
        ("Weighted Precision", "Precision", "Blues"),
        ("Weighted Recall", "Recall", "Greens"),
        ("Weighted F1-Score", "F1-Score", "coolwarm"),
    ]

    for ax, (metric_key, ylabel, palette) in zip(axes, metric_plots):
        score_dict = metric_scores.get(metric_key, {})
        names = list(score_dict.keys())
        values = list(score_dict.values())

        sns.barplot(x=names, y=values, hue=names, dodge=False, palette=palette,␣
↪legend=False, ax=ax)
        ax.set_title(f"{phase_label}: {metric_key}", fontsize=12,␣
↪fontweight="bold")
        ax.set_ylabel(ylabel, fontsize=11)
        ax.set_xlabel("Model", fontsize=11)
        ax.tick_params(axis="x", rotation=25)
        ax.set_ylim(0, max(values) + 0.1 if values else 1.0)

        for i, v in enumerate(values):
            ax.text(i, v + 0.005, f"{v:.4f}", ha="center", fontsize=10,␣
↪fontweight="bold")

    plt.tight_layout()
    plt.show()
```

5.2.13 Train & Evaluate Pipeline

[21]:
```
core_models = [
    (CNNSmall,    "cnn_10"),
    (CNNLarge,    "cnn_18"),
    (ResNetSmall, "resnet_10"),
    (ResNetLarge, "resnet_18")
]
model_names = [name for _, name in core_models]


def train_and_evaluate_phase(phase_label, weights_prefix, model_kwargs,
                             train_ldr=None, val_ldr=None, debug=False,␣
 ↪epochs=21):
    """Train (or load) all core models for one technique phase, compute␣
 ↪metrics, and return histories + metrics list.

    Each technique is applied in isolation: model_kwargs should only enable the␣
 ↪flags
    relevant to that specific technique (e.g., use_batch_norm=True for the BN␣
 ↪phase).

    Returns (histories_dict, metrics_list).
    """
    if train_ldr is None: train_ldr = train_loader
    if val_ldr is None: val_ldr = val_loader

    histories = {}
    metrics_list = []

    print(f"INITIATING {phase_label.upper()} TRAINING...\n")

    for model_class, model_name in core_models:
        weights_file = f"{weights_prefix}/{weights_prefix}_{model_name}.pth"
        model = model_class(**model_kwargs).to(device)

        history = load_weights(model, weights_file, device)
        if history is not None:
            print(f"--- {model_name} ({phase_label}) loaded. Skipping training.␣
 ↪---")
        else:
            print(f"--- Training {model_name} ({phase_label}) ---")
            criterion = nn.CrossEntropyLoss()
            optimizer = optim.Adam(model.parameters(), lr=0.001)
            history = train_model(model, train_ldr, val_ldr, criterion,␣
 ↪optimizer,
                                  epochs=epochs, device=device, debug=debug)
```

```
            save_weights(model, weights_file, history)
            print(f"--- Training {model_name} Completed ---")

        histories[model_name] = history

        # compute/load metrics
        metrics_key = f"{weights_prefix}_{model_name}"
        precision, recall, f1, top1_acc, top5_acc, top_confused, y_true, y_pred␣
↪= \
            load_or_compute_metrics(metrics_key, model, val_ldr, full_dataset.
↪classes, device)

        # plot confusion matrix only if freshly computed
        if y_true is not None:
            plot_confusion_matrix(y_true, y_pred, class_names=full_dataset.
↪classes,
                                  model_name=f"{model_name} ({phase_label})")

        metrics_list.append({
            "Model Name": model_name,
            "Weighted Precision": precision,
            "Weighted Recall": recall,
            "Weighted F1-Score": f1,
        })

        print(f"{model_name} Processed | F1: {f1:.4f} | Top-1: {top1_acc:.4f} |␣
↪Top-5: {top5_acc:.4f}\n")

    print(f"ALL {phase_label.upper()} MODELS COMPLETE.")
    return histories, metrics_list
```

5.2.14 Create Model Metric Table

[22]:
```
def plot_phase_results(histories, metrics_list, phase_label):
    """Plot all comparison charts and F-score bars for a training phase."""
    # loss comparison
    plot_loss_comparison(histories, model_names, title=f"{phase_label}:␣
↪Training and Validation Loss")

    # accuracy comparisons
    plot_accuracy_comparison(histories, model_names, acc_key='val_acc',
                             ylabel='Top-1 Accuracy', title=f"{phase_label}:␣
↪Top-1 Validation Accuracy")
    plot_accuracy_comparison(histories, model_names, acc_key='val_top5_acc',
                             ylabel='Top-5 Accuracy', title=f"{phase_label}:␣
↪Top-5 Validation Accuracy")
```

```
    # F-score bars
    metric_scores = {"Weighted Precision": {}, "Weighted Recall": {}, "Weighted␣
↪F1-Score": {}}
    for entry in metrics_list:
        name = entry["Model Name"]
        metric_scores["Weighted Precision"][name] = entry["Weighted Precision"]
        metric_scores["Weighted Recall"][name] = entry["Weighted Recall"]
        metric_scores["Weighted F1-Score"][name] = entry["Weighted F1-Score"]

    results_df = pd.DataFrame(metrics_list)
    results_df.insert(1, "Phase", phase_label)
    display(results_df)
    plot_f_score(metric_scores, phase_label=phase_label)
```

5.2.15 Debug Smoke Test

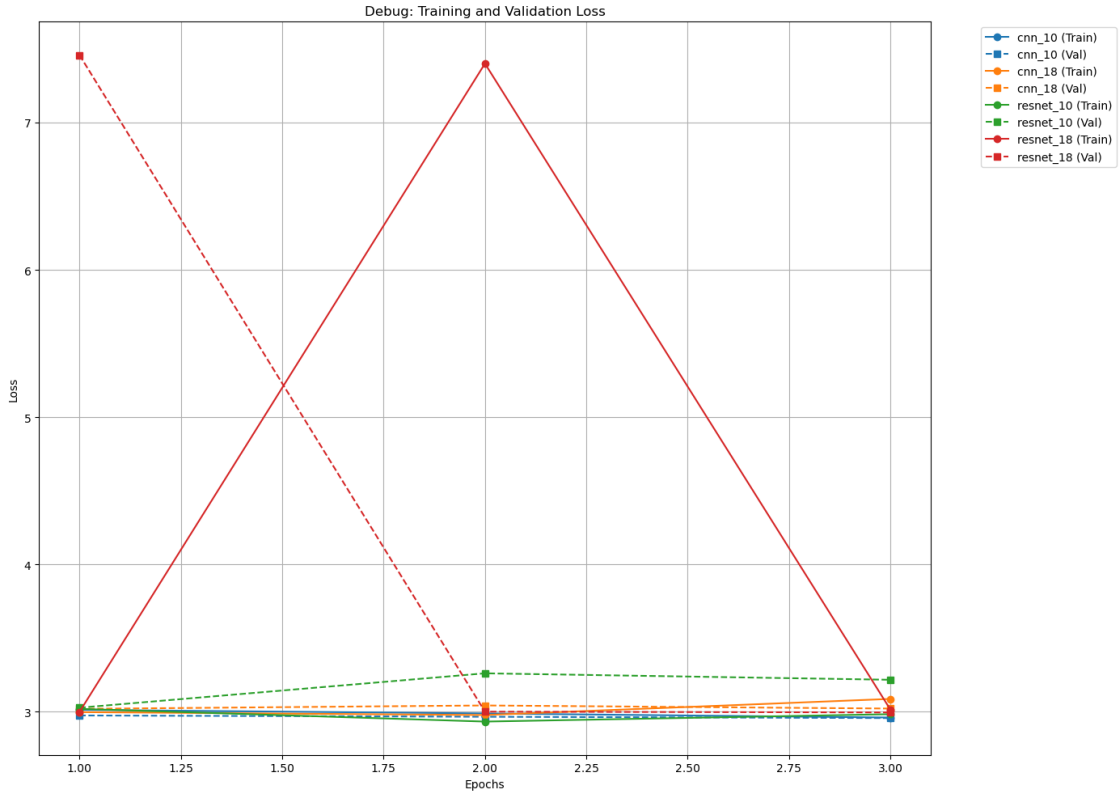
Run this to ensure the code is working properly. ____

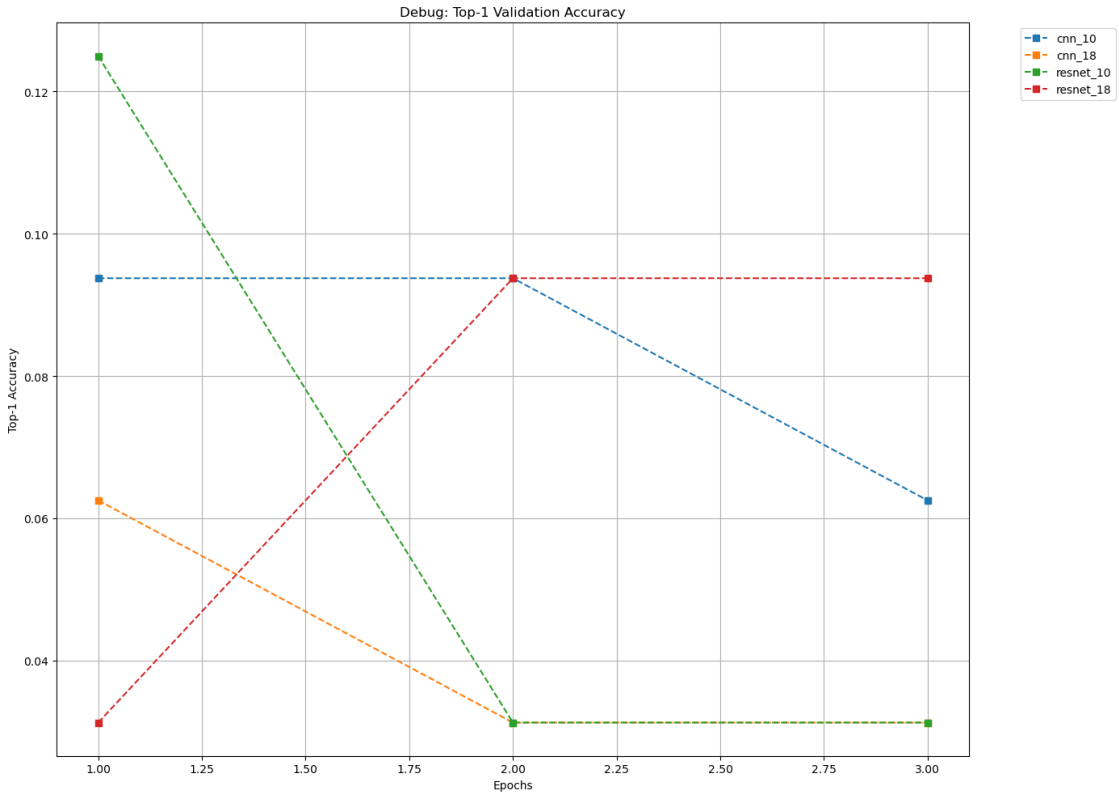
[23]:
```
debug_histories, debug_metrics = train_and_evaluate_phase(
    phase_label="Debug",
    weights_prefix="debug",
    model_kwargs={"use_batch_norm": False, "use_dropout": False, "num_classes":␣
↪20},
    debug=True
)
```

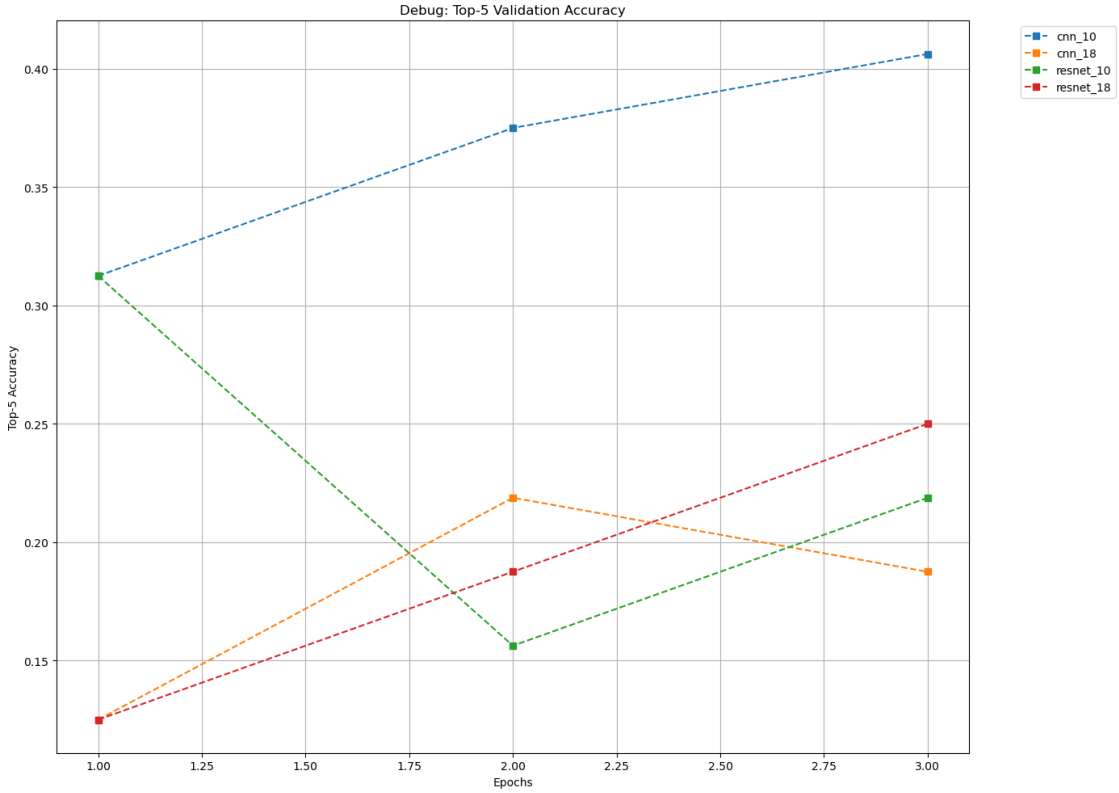
```
INITIATING DEBUG TRAINING…

Loaded weights from model_weights\debug/debug_cnn_10.pth
--- cnn_10 (Debug) loaded. Skipping training. ---
Loaded existing metrics for debug_cnn_10
cnn_10 Processed | F1: 0.0107 | Top-1: 0.0470 | Top-5: 0.2600

Loaded weights from model_weights\debug/debug_cnn_18.pth
--- cnn_18 (Debug) loaded. Skipping training. ---
Loaded existing metrics for debug_cnn_18
cnn_18 Processed | F1: 0.0101 | Top-1: 0.0555 | Top-5: 0.2425

Loaded weights from model_weights\debug/debug_resnet_10.pth
--- resnet_10 (Debug) loaded. Skipping training. ---
Loaded existing metrics for debug_resnet_10
resnet_10 Processed | F1: 0.0076 | Top-1: 0.0495 | Top-5: 0.2530

Loaded weights from model_weights\debug/debug_resnet_18.pth
--- resnet_18 (Debug) loaded. Skipping training. ---
Loaded existing metrics for debug_resnet_18
```

```
resnet_18 Processed | F1: 0.0048 | Top-1: 0.0500 | Top-5: 0.2505

ALL DEBUG MODELS COMPLETE.
```

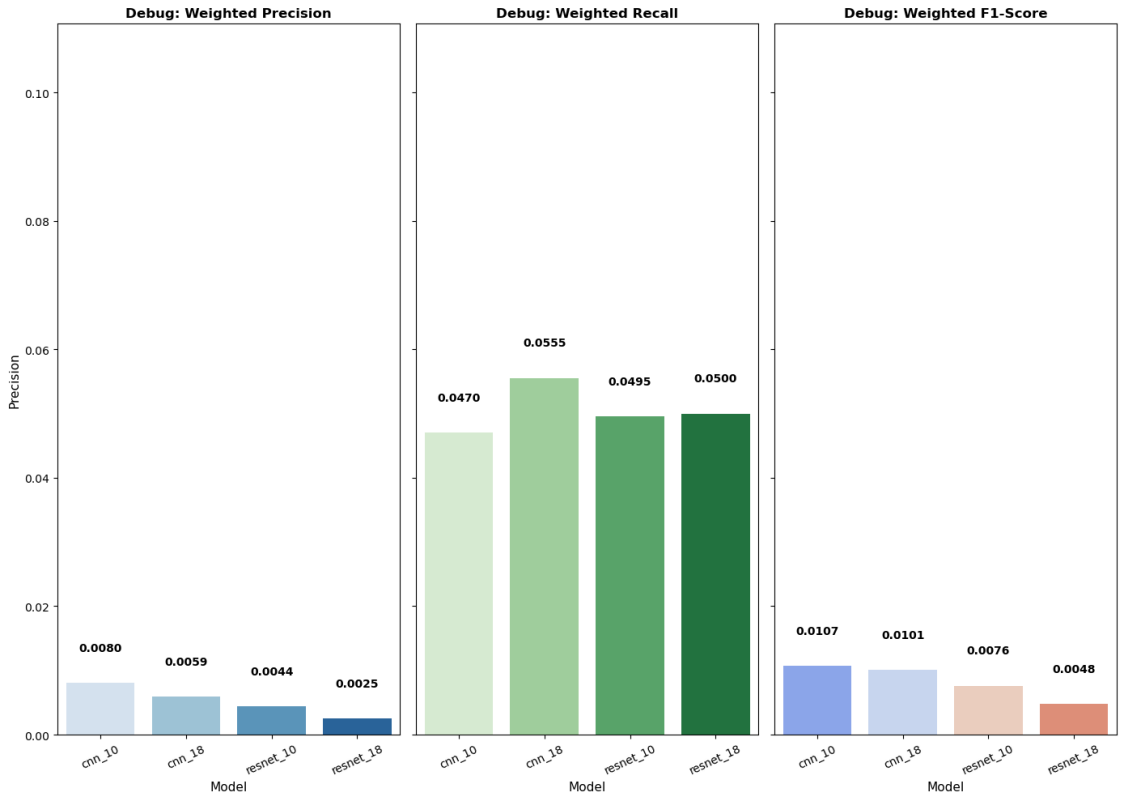
[24]:
```
plot_phase_results(debug_histories, debug_metrics, "Debug")
```




Debug: Top-1 Validation Accuracy
Top-1 Accuracy
Epochs
0.12
0.10
0.08
0.06
0.04
1.00
1.25
1.50
1.75
2.00
2.25
2.50
2.75
3.00
cnn_10
cnn_18
resnet_10
resnet_18


Debug: Top-5 Validation Accuracy
Top-5 Accuracy
Epochs
0.40
0.35
0.30
0.25
0.20
0.15
1.00
1.25
1.50
1.75
2.00
2.25
2.50
2.75
3.00
cnn_10
cnn_18
resnet_10
resnet_18

	Model Name	Phase	Weighted Precision	Weighted Recall	Weighted F1-Score
0	cnn_10	Debug	0.008018	0.0470	0.010678
1	cnn_18	Debug	0.005885	0.0555	0.010065
2	resnet_10	Debug	0.004378	0.0495	0.007631
3	resnet_18	Debug	0.002500	0.0500	0.004762



5.3 (5.1) Baseline Models

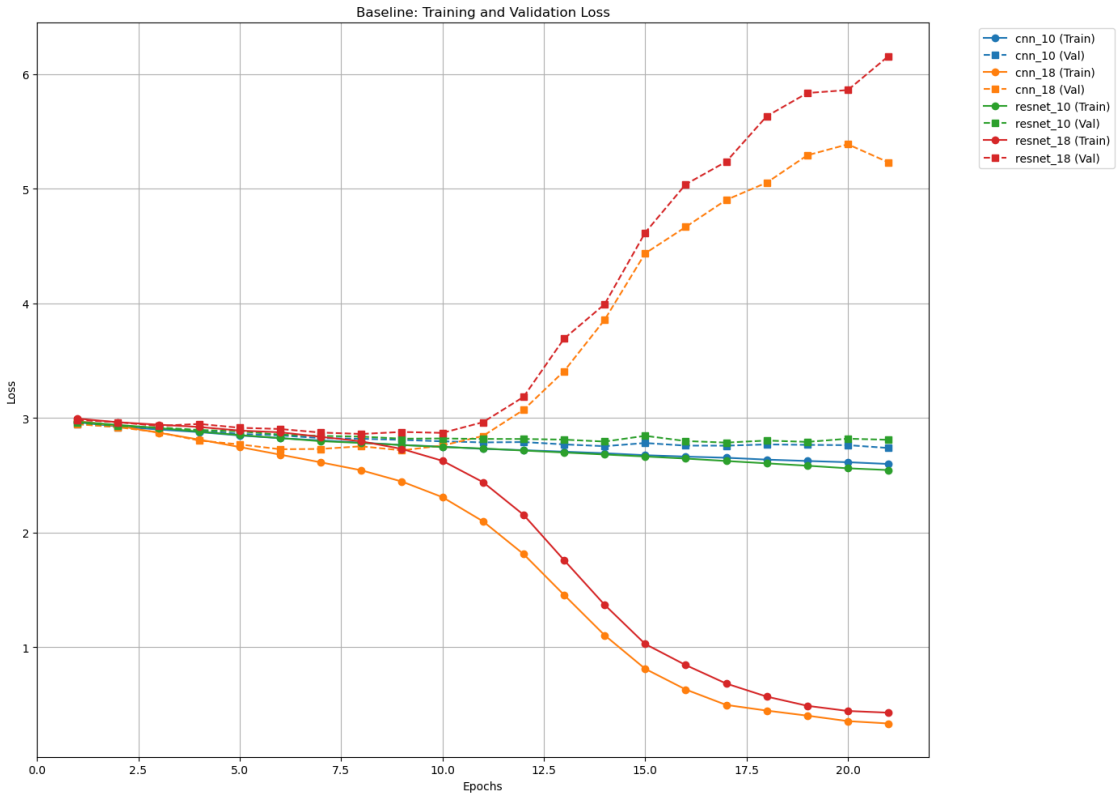
The baseline phase trains all 4 architectures (cnn_10, cnn_18, resnet_10, resnet_18) with no performance improvement techniques enabled. No batch normalization, no dropout, and no data augmentation. This establishes a reference point so each technique's individual impact can be measured against a clean starting point.

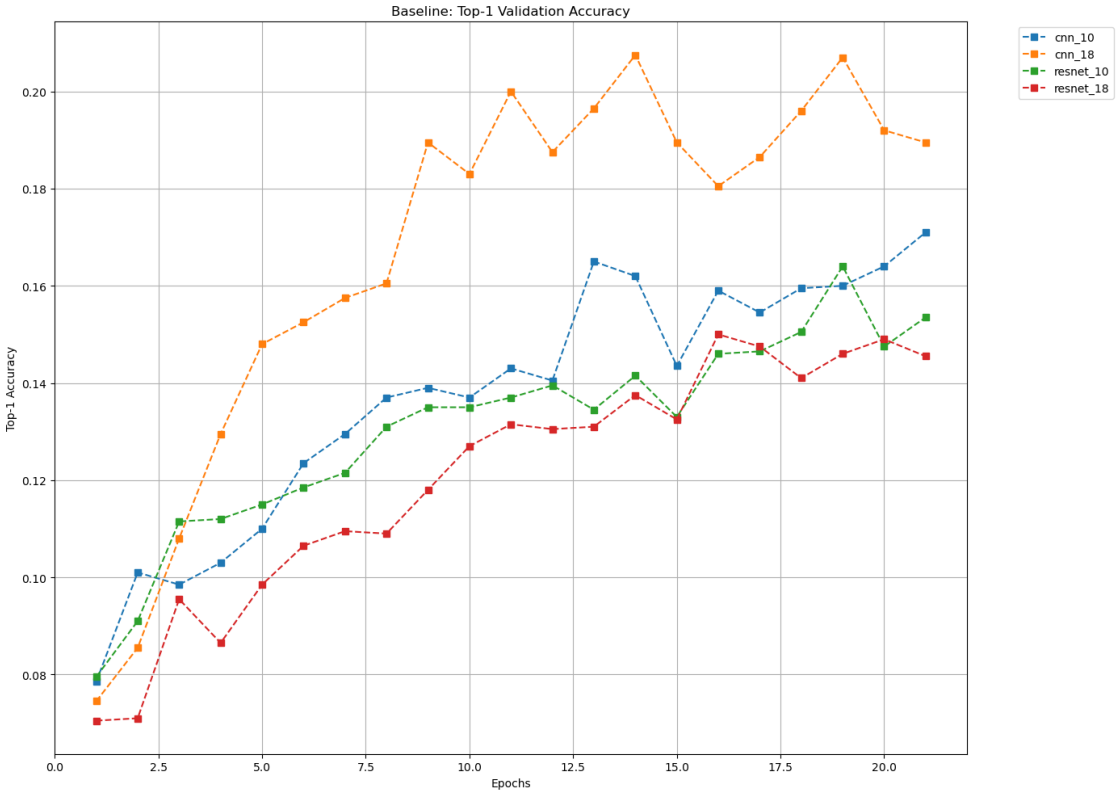
The proposal originally named CNN-18 as the sole baseline. I expanded this to include all 4 models at baseline so I could track how each architecture responds to each technique independently.

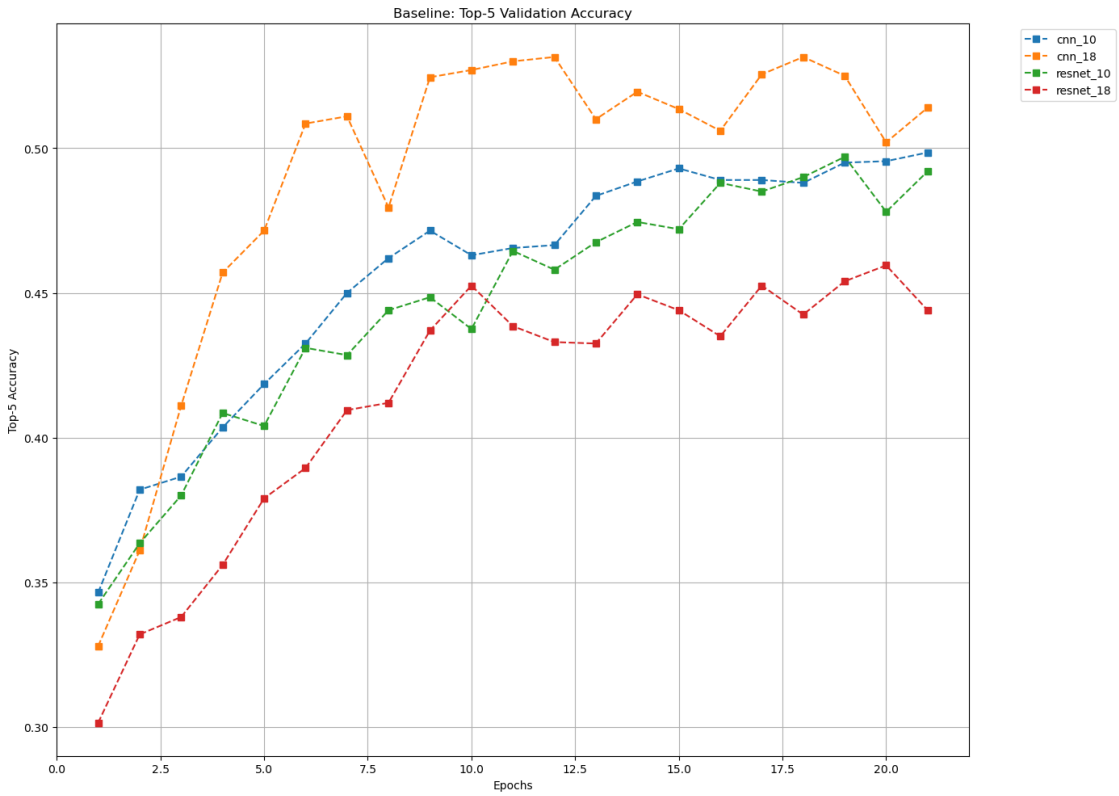
[25]:
```
baseline_histories, baseline_metrics = train_and_evaluate_phase(
    phase_label="Baseline",
    weights_prefix="baseline",
    model_kwargs={"use_batch_norm": False, "use_dropout": False, "num_classes":␣
 ↪20}
)
```

```
INITIATING BASELINE TRAINING…

Loaded weights from model_weights\baseline/baseline_cnn_10.pth
--- cnn_10 (Baseline) loaded. Skipping training. ---
Loaded existing metrics for baseline_cnn_10
cnn_10 Processed | F1: 0.1548 | Top-1: 0.1710 | Top-5: 0.4985

Loaded weights from model_weights\baseline/baseline_cnn_18.pth
--- cnn_18 (Baseline) loaded. Skipping training. ---
Loaded existing metrics for baseline_cnn_18
cnn_18 Processed | F1: 0.1681 | Top-1: 0.1895 | Top-5: 0.5245

Loaded weights from model_weights\baseline/baseline_resnet_10.pth
--- resnet_10 (Baseline) loaded. Skipping training. ---
Loaded existing metrics for baseline_resnet_10
resnet_10 Processed | F1: 0.1371 | Top-1: 0.1465 | Top-5: 0.4845

Loaded weights from model_weights\baseline/baseline_resnet_18.pth
--- resnet_18 (Baseline) loaded. Skipping training. ---
Loaded existing metrics for baseline_resnet_18
resnet_18 Processed | F1: 0.0862 | Top-1: 0.1090 | Top-5: 0.4120

ALL BASELINE MODELS COMPLETE.
```

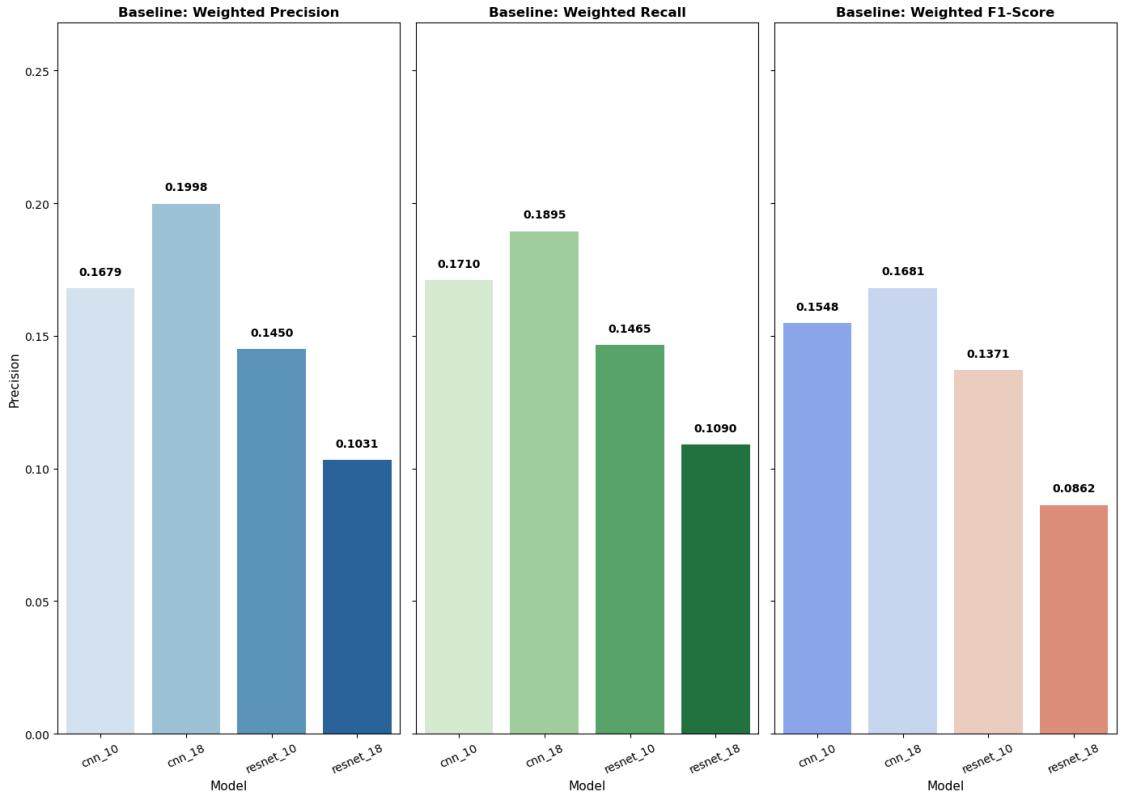
[26]:
```
plot_phase_results(baseline_histories, baseline_metrics, "Baseline")
```


Baseline: Training and Validation Loss
Loss
Epochs
cnn_10 (Train)
cnn_10 (Val)
cnn_18 (Train)
cnn_18 (Val)
resnet_10 (Train)
resnet_10 (Val)
resnet_18 (Train)
resnet_18 (Val)


Baseline: Top-1 Validation Accuracy
Top-1 Accuracy
Epochs
cnn_10
cnn_18
resnet_10
resnet_18



	Model Name	Phase	Weighted Precision	Weighted Recall	Weighted F1-Score
0	cnn_10	Baseline	0.167924	0.1710	0.154777
1	cnn_18	Baseline	0.199849	0.1895	0.168056
2	resnet_10	Baseline	0.144964	0.1465	0.137130
3	resnet_18	Baseline	0.103118	0.1090	0.086242



5.3.1 Evaluate Baseline Results

The large models are overfitting. Training loss drops to near 0 while validation loss keeps climbing, especially after epoch 10. The large ResNet has the lowest training loss but the highest validation loss; it memorized the training data so well that it started getting worse on unseen data. This is a clear sign the model needs regularization.

The small models are underfitting. They dont have enough layers to capture what the larger models are learning, so their loss curves stay relatively flat throughout training. The small CNN has the lowest validation loss out of all models at baseline.

Baseline ranking (best to worst by validation loss): 1. CNN-10 (small CNN) 2. ResNet-10 (small ResNet) 3. CNN-18 (large CNN) 4. ResNet-18 (large ResNet)

I expect these rankings to change after applying techniques. The deeper models should benefit the most from normalization and regularization since those directly address the overfitting and gradient problems they have.

Techniques I will evaluate (each applied in isolation so I can measure individual impact): 1. Batch Normalization - stabilize internal activations 2. Data Augmentation - increase effective dataset size/diversity 3. Dropout Regularization - force the model to build redundant feature representations

Note: early stopping (patience = 5) is built into the training loop for all experiments including baseline, so it is not counted as a separate technique. It monitors validation loss each epoch and

saves the model weights whenever validation loss reaches a new low. If validation loss does not improve for 5 consecutive epochs, training stops and the best weights are restored. This means all evaluation is done on the weights with the lowest validation loss rather than whatever the model looked like at the end of training. For the overfitting models like ResNet-18 where validation loss starts climbing after a few epochs, this is the difference between evaluating a reasonable model and evaluating one that has degraded significantly.

I chose to test each technique in isolation rather than stacking them on top of each other. This way I can attribute a performance change to the specific technique being tested rather than guessing which one in the stack caused it.

---

6 (6.) Performance Improvement Techniques

6.1 Technique 1 : Batch Normalization

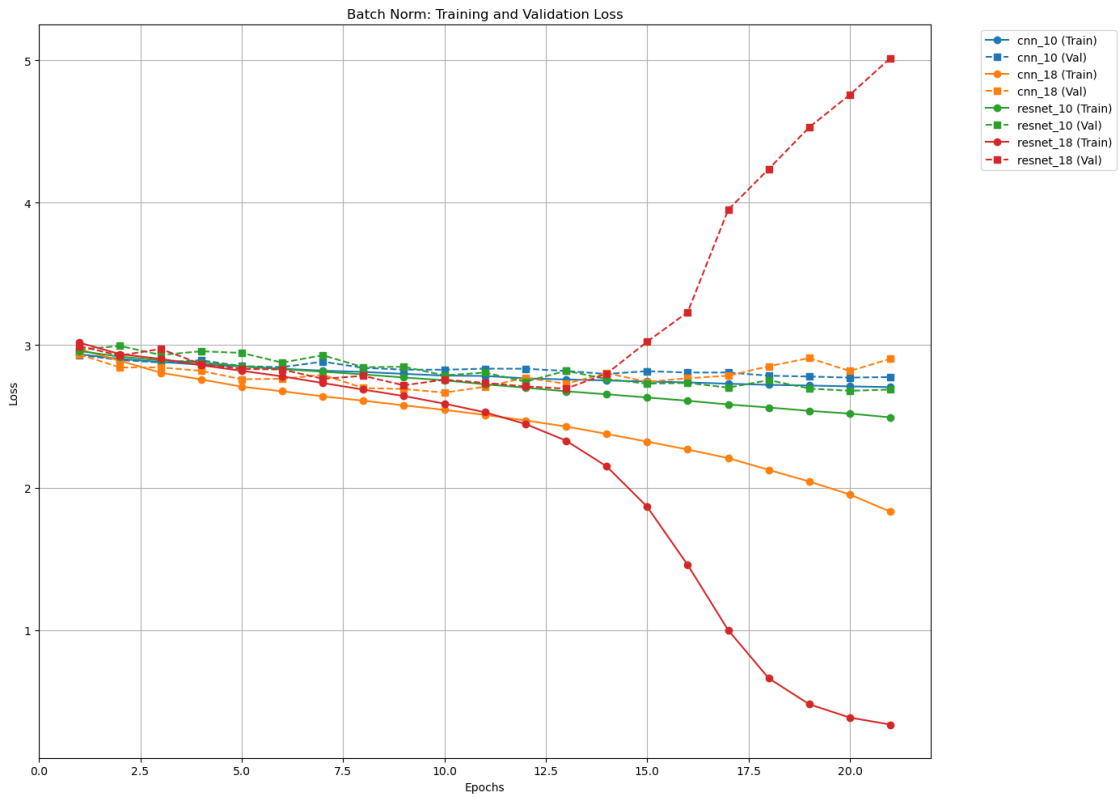
---

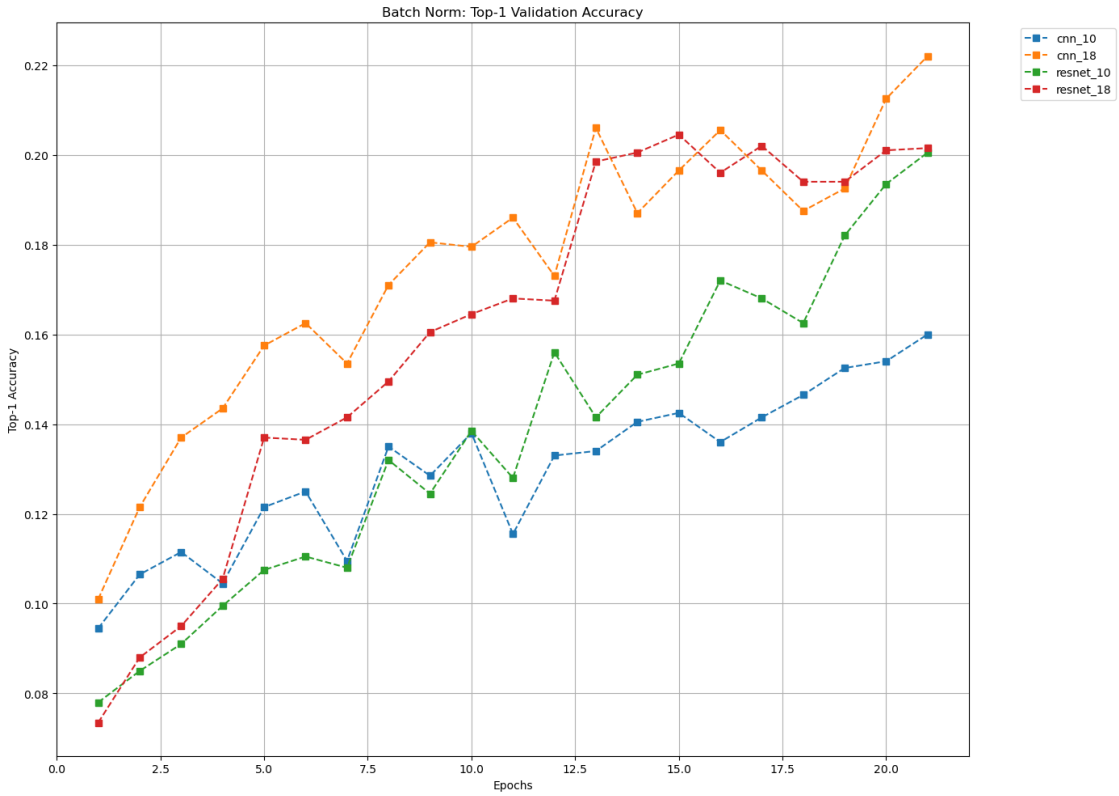
[27]:
```
bn_histories, bn_metrics = train_and_evaluate_phase(
    phase_label="Batch Norm",
    weights_prefix="batchnorm",
    model_kwargs={"use_batch_norm": True, "use_dropout": False, "num_classes":␣
 ↪20}
)
```

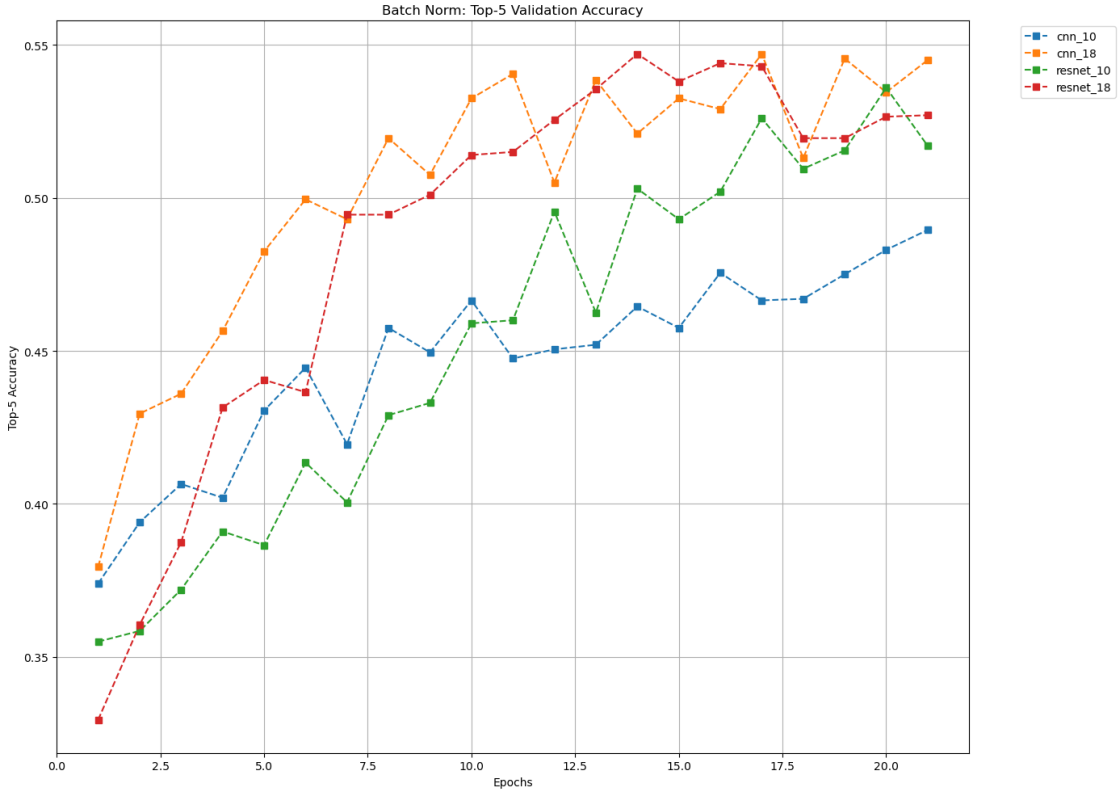
```
INITIATING BATCH NORM TRAINING…

Loaded weights from model_weights\batchnorm/batchnorm_cnn_10.pth
--- cnn_10 (Batch Norm) loaded. Skipping training. ---
Loaded existing metrics for batchnorm_cnn_10
cnn_10 Processed | F1: 0.1315 | Top-1: 0.1540 | Top-5: 0.4830

Loaded weights from model_weights\batchnorm/batchnorm_cnn_18.pth
--- cnn_18 (Batch Norm) loaded. Skipping training. ---
Loaded existing metrics for batchnorm_cnn_18
cnn_18 Processed | F1: 0.1520 | Top-1: 0.1795 | Top-5: 0.5325

Loaded weights from model_weights\batchnorm/batchnorm_resnet_10.pth
--- resnet_10 (Batch Norm) loaded. Skipping training. ---
Loaded existing metrics for batchnorm_resnet_10
resnet_10 Processed | F1: 0.1765 | Top-1: 0.1935 | Top-5: 0.5360

Loaded weights from model_weights\batchnorm/batchnorm_resnet_18.pth
--- resnet_18 (Batch Norm) loaded. Skipping training. ---
Loaded existing metrics for batchnorm_resnet_18
```

```
resnet_18 Processed | F1: 0.1845 | Top-1: 0.1985 | Top-5: 0.5355

ALL BATCH NORM MODELS COMPLETE.
```

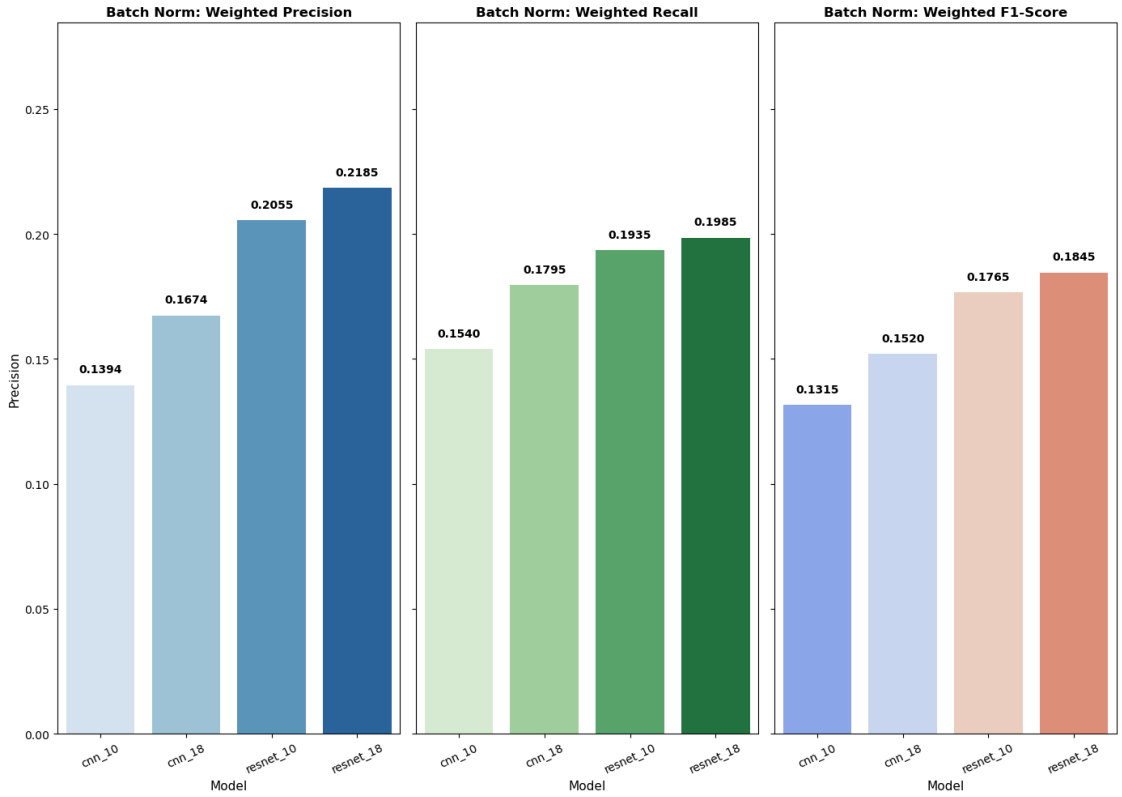
[28]:
```
plot_phase_results(bn_histories, bn_metrics, "Batch Norm")
```




Batch Norm: Top-1 Validation Accuracy
Top-1 Accuracy
Epochs
cnn_10
cnn_18
resnet_10
resnet_18


Batch Norm: Top-5 Validation Accuracy
Top-5 Accuracy
Epochs
cnn_10
cnn_18
resnet_10
resnet_18

```
  Model Name       Phase  Weighted Precision  Weighted Recall  \
0     cnn_10  Batch Norm            0.139439           0.1540
1     cnn_18  Batch Norm            0.167364           0.1795
2  resnet_10  Batch Norm            0.205456           0.1935
3  resnet_18  Batch Norm            0.218453           0.1985

   Weighted F1-Score
0           0.131537
1           0.152006
2           0.176521
3           0.184498
```



6.1.1 Evaluate Batch Normalization Results

Batch normalization normalizes activations after each convolutional layer to zero mean and unit variance. This stabilizes what each layer receives as input so the optimizer can take larger, more consistent steps without activations exploding or vanishing.

ResNet-18 saw the biggest improvement, jumping from 10.9% to 19.85% top-1 accuracy. At baseline it was the worst model; with batch norm it becomes the best. This makes sense because deeper

networks suffer more from the vanishing gradient problem, and normalization directly addresses that by keeping activations in a stable range throughout the forward pass.

CNN-18 went from 18.95% to 17.95% which is slightly worse. The batch norm overhead on a model that was already learning reasonably well at baseline didnt help here.

The small CNN got slightly worse with batch norm (15.4% vs 17.1% baseline). For a shallow 3-block network the gradients were already stable enough without normalization, so the added parameters just introduced noise.

Looking at the loss curves, batch-norm models converge faster and smoother than their baseline versions. The validation loss curves are less erratic which is consistent with reducing internal covariate shift.

Batch normalization is most valuable for deeper architectures where gradient flow is the bottleneck. For shallow models it adds overhead without meaningful benefit.

6.2 Technique 2 : Data Augmentation

---

[29]:
```
aug_transforms = transforms.Compose([
    transforms.RandomResizedCrop(224, scale=(0.8, 1.0)),
    transforms.RandomHorizontalFlip(p=0.5),
    transforms.RandomRotation(degrees=15),
    transforms.ToTensor(),
    transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
])

full_dataset_aug = datasets.ImageFolder(data_dir, transform=aug_transforms)
aug_train_dataset = Subset(full_dataset_aug, train_dataset.indices)

aug_train_loader = DataLoader(
    aug_train_dataset,
    batch_size=BATCH_SIZE,
    shuffle=True,
    num_workers=NUM_WORKERS,
    pin_memory=True
)

print(f"Augmented training loader created with {len(aug_train_dataset)} images.
 ↪")
```

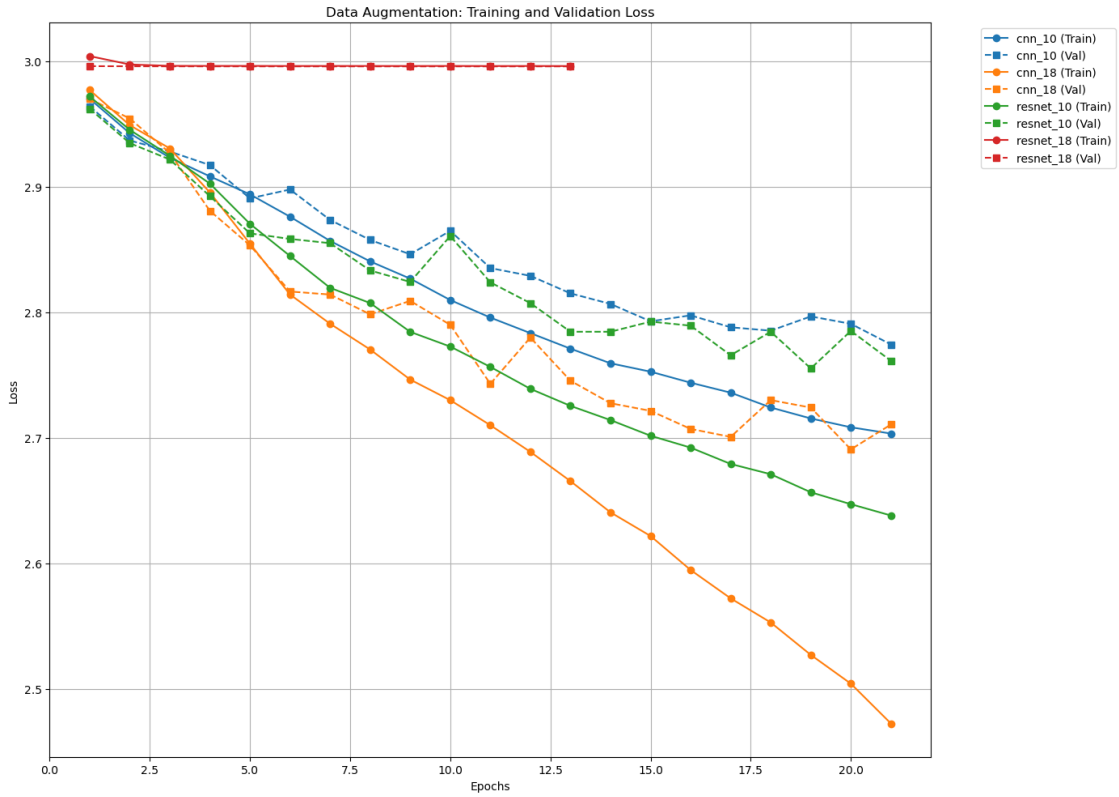
```
Augmented training loader created with 16000 images.
```

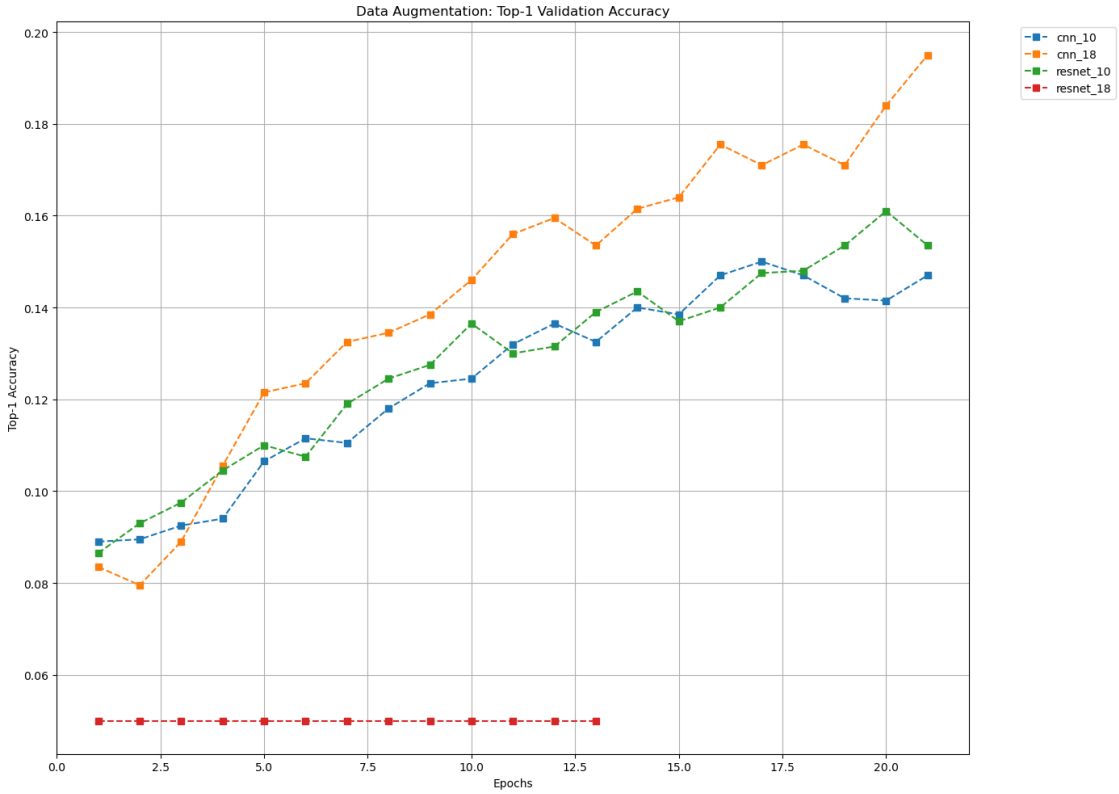
[30]:
```
aug_histories, aug_metrics = train_and_evaluate_phase(
    phase_label="Data Augmentation",
    weights_prefix="data_aug",
```

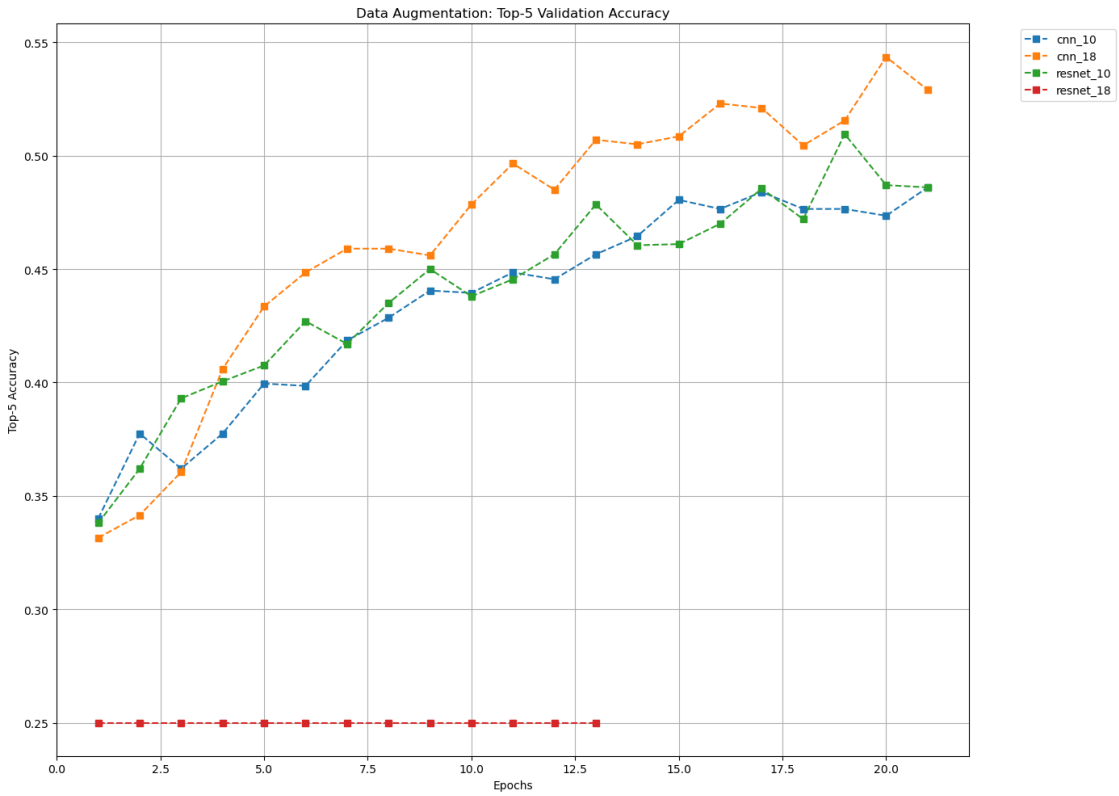
```
    model_kwargs={"use_batch_norm": False, "use_dropout": False, "num_classes":␣
↪20},
    train_ldr=aug_train_loader
)
```

```
INITIATING DATA AUGMENTATION TRAINING…

Loaded weights from model_weights\data_aug/data_aug_cnn_10.pth
--- cnn_10 (Data Augmentation) loaded. Skipping training. ---
Loaded existing metrics for data_aug_cnn_10
cnn_10 Processed | F1: 0.1349 | Top-1: 0.1470 | Top-5: 0.4860

Loaded weights from model_weights\data_aug/data_aug_cnn_18.pth
--- cnn_18 (Data Augmentation) loaded. Skipping training. ---
Loaded existing metrics for data_aug_cnn_18
cnn_18 Processed | F1: 0.1697 | Top-1: 0.1840 | Top-5: 0.5440

Loaded weights from model_weights\data_aug/data_aug_resnet_10.pth
--- resnet_10 (Data Augmentation) loaded. Skipping training. ---
Loaded existing metrics for data_aug_resnet_10
resnet_10 Processed | F1: 0.1313 | Top-1: 0.1535 | Top-5: 0.5095

Loaded weights from model_weights\data_aug/data_aug_resnet_18.pth
--- resnet_18 (Data Augmentation) loaded. Skipping training. ---
Loaded existing metrics for data_aug_resnet_18
resnet_18 Processed | F1: 0.0048 | Top-1: 0.0500 | Top-5: 0.2500

ALL DATA AUGMENTATION MODELS COMPLETE.
```

[31]:
```
plot_phase_results(aug_histories, aug_metrics, "Data Augmentation")
```

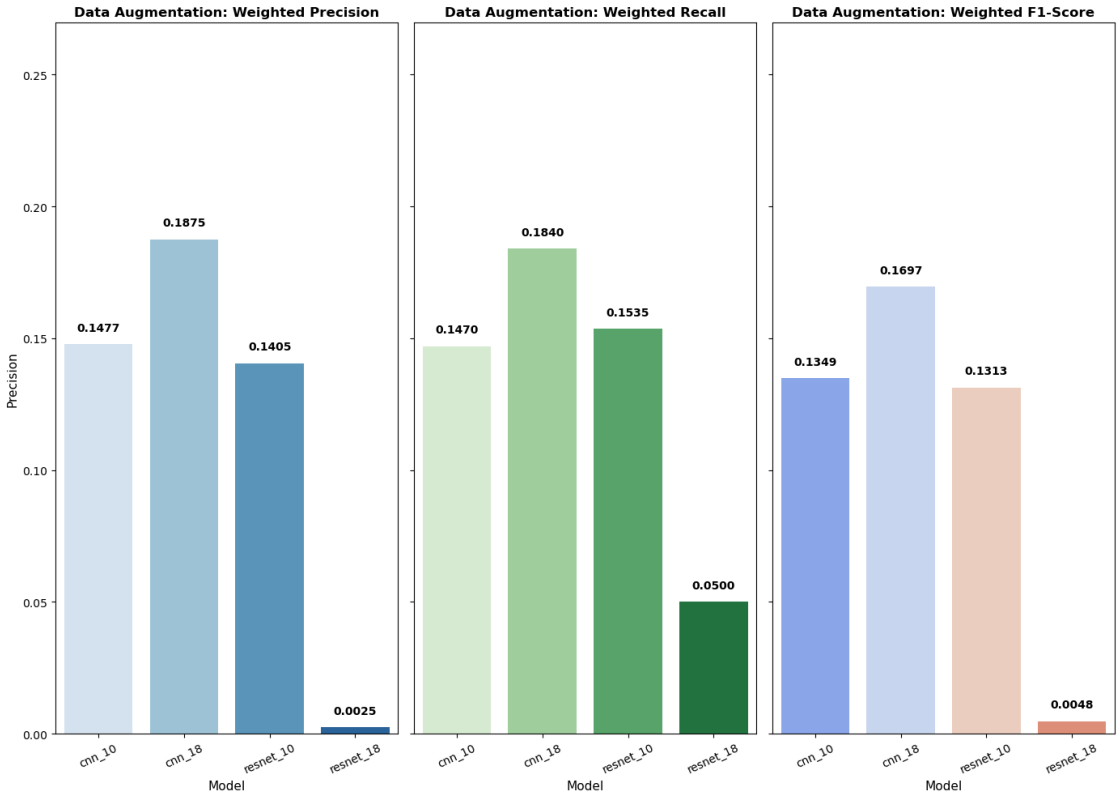

Data Augmentation: Training and Validation Loss
Loss
Epochs
cnn_10 (Train)
cnn_10 (Val)
cnn_18 (Train)
cnn_18 (Val)
resnet_10 (Train)
resnet_10 (Val)
resnet_18 (Train)
resnet_18 (Val)


Data Augmentation: Top-1 Validation Accuracy
Top-1 Accuracy
Epochs
cnn_10
cnn_18
resnet_10
resnet_18



	Model Name	Phase	Weighted Precision	Weighted Recall	\
0	cnn_10	Data Augmentation	0.147706	0.1470	
1	cnn_18	Data Augmentation	0.187527	0.1840	
2	resnet_10	Data Augmentation	0.140506	0.1535	
3	resnet_18	Data Augmentation	0.002500	0.0500	

	Weighted F1-Score
0	0.134898
1	0.169670
2	0.131331
3	0.004762



6.2.1 Evaluate Data Augmentation Results

Data augmentation applies random crops (scale 0.8-1.0), horizontal flips (p=0.5), and rotations (up to 15 degrees) to training images. The model sees slightly different versions of each image every epoch which should reduce overfitting by increasing effective dataset diversity. Augmentation is only applied to the training set; validation and test sets use the standard transform pipeline.

CNN-18 benefited the most. Its top-1 accuracy is comparable to baseline (18.4% vs 18.95%) and its top-5 accuracy went up slightly (54.4% vs 52.45%). The augmented data helped the deeper CNN generalize without needing normalization layers.

ResNet-18 collapsed entirely to 5% top-1 accuracy which is random guessing across 20 classes. The model predicts "Grime" for every input. This is mode collapse; training a deep residual network on varied augmented data without batch normalization pushes the gradients into a degenerate solution. Data augmentation alone is not viable for deep architectures without normalization to keep activations stable.

CNN-10 and ResNet-10 showed mixed results with accuracy slightly below baseline. The shallow models dont have enough capacity to benefit from increased data variety; the augmentation just adds noise they cant learn through.

Data augmentation does not uniformly improve performance. It can be destructive for deep models without batch normalization. The ResNet-18 collapse demonstrates why technique interactions matter even when testing in isolation.

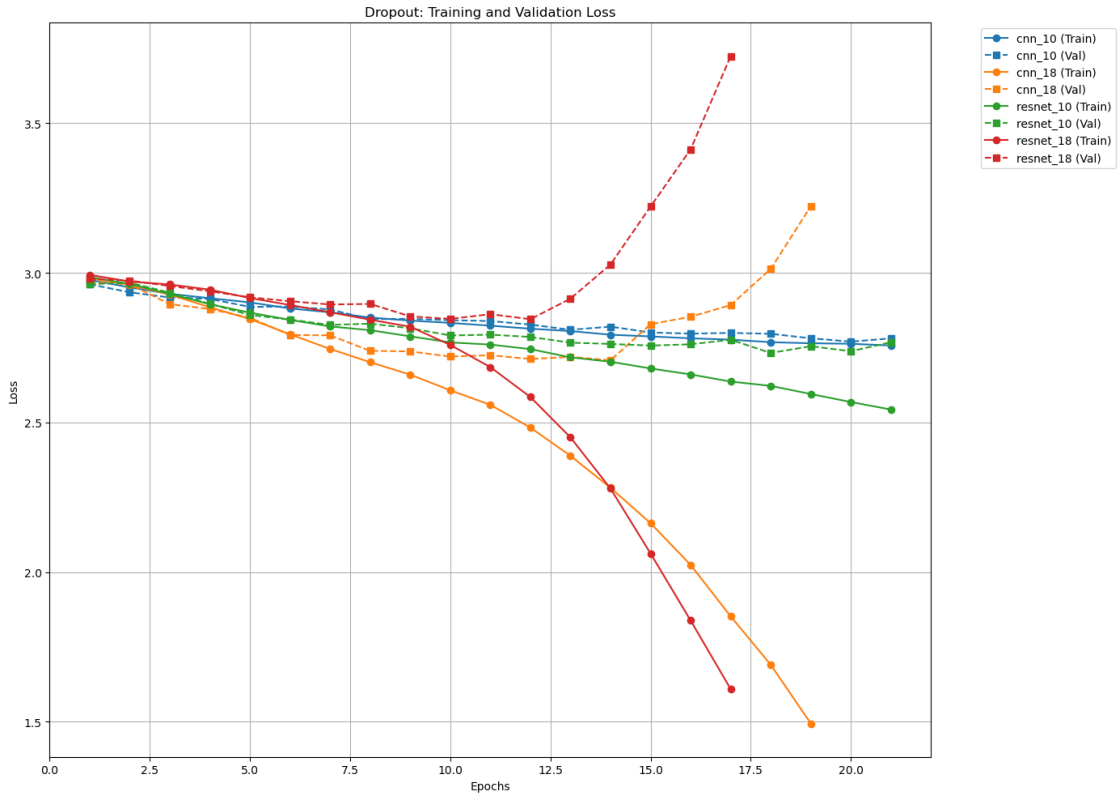
6.3 Technique 3 : Dropout Regularization

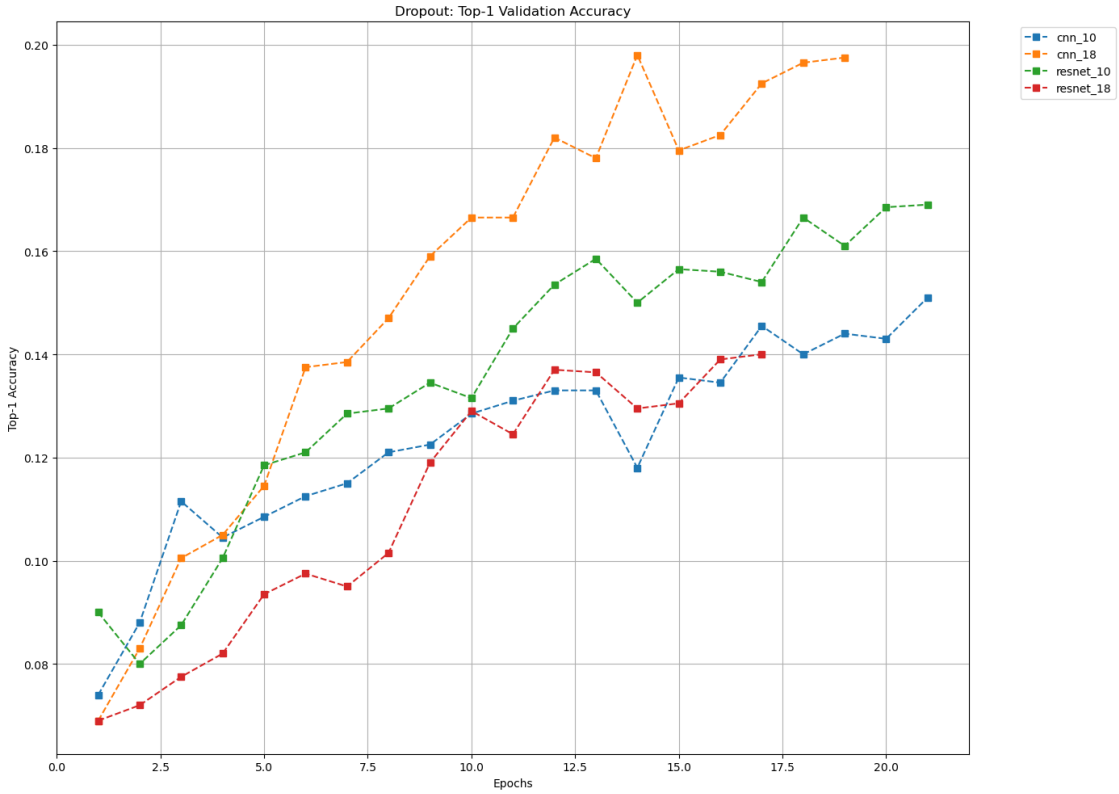
---

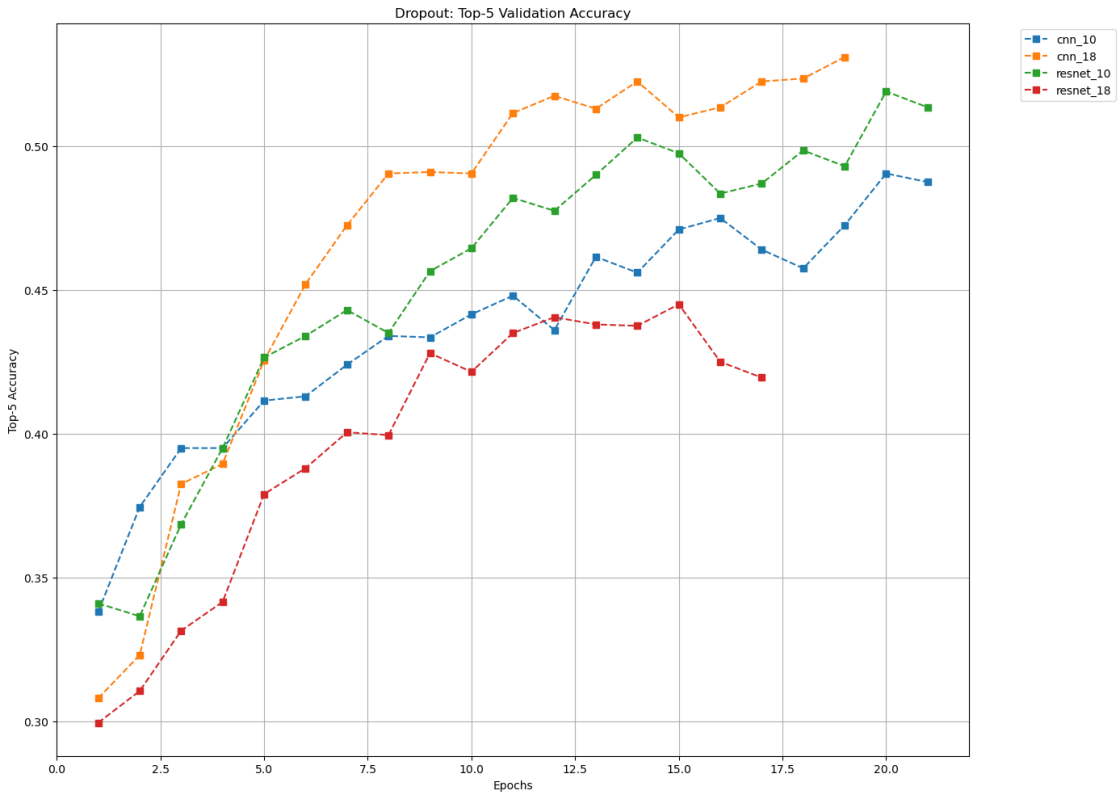
[32]:
```
dropout_histories, dropout_metrics = train_and_evaluate_phase(
    phase_label="Dropout",
    weights_prefix="dropout",
    model_kwargs={"use_batch_norm": False, "use_dropout": True, "num_classes":␣
 ↪20}
)
```

```
INITIATING DROPOUT TRAINING…

Loaded weights from model_weights\dropout/dropout_cnn_10.pth
--- cnn_10 (Dropout) loaded. Skipping training. ---
Loaded existing metrics for dropout_cnn_10
cnn_10 Processed | F1: 0.1222 | Top-1: 0.1430 | Top-5: 0.4905

Loaded weights from model_weights\dropout/dropout_cnn_18.pth
--- cnn_18 (Dropout) loaded. Skipping training. ---
Loaded existing metrics for dropout_cnn_18
cnn_18 Processed | F1: 0.1891 | Top-1: 0.1980 | Top-5: 0.5230

Loaded weights from model_weights\dropout/dropout_resnet_10.pth
--- resnet_10 (Dropout) loaded. Skipping training. ---
Loaded existing metrics for dropout_resnet_10
resnet_10 Processed | F1: 0.1485 | Top-1: 0.1665 | Top-5: 0.4985

Loaded weights from model_weights\dropout/dropout_resnet_18.pth
--- resnet_18 (Dropout) loaded. Skipping training. ---
Loaded existing metrics for dropout_resnet_18
resnet_18 Processed | F1: 0.1267 | Top-1: 0.1370 | Top-5: 0.4405

ALL DROPOUT MODELS COMPLETE.
```

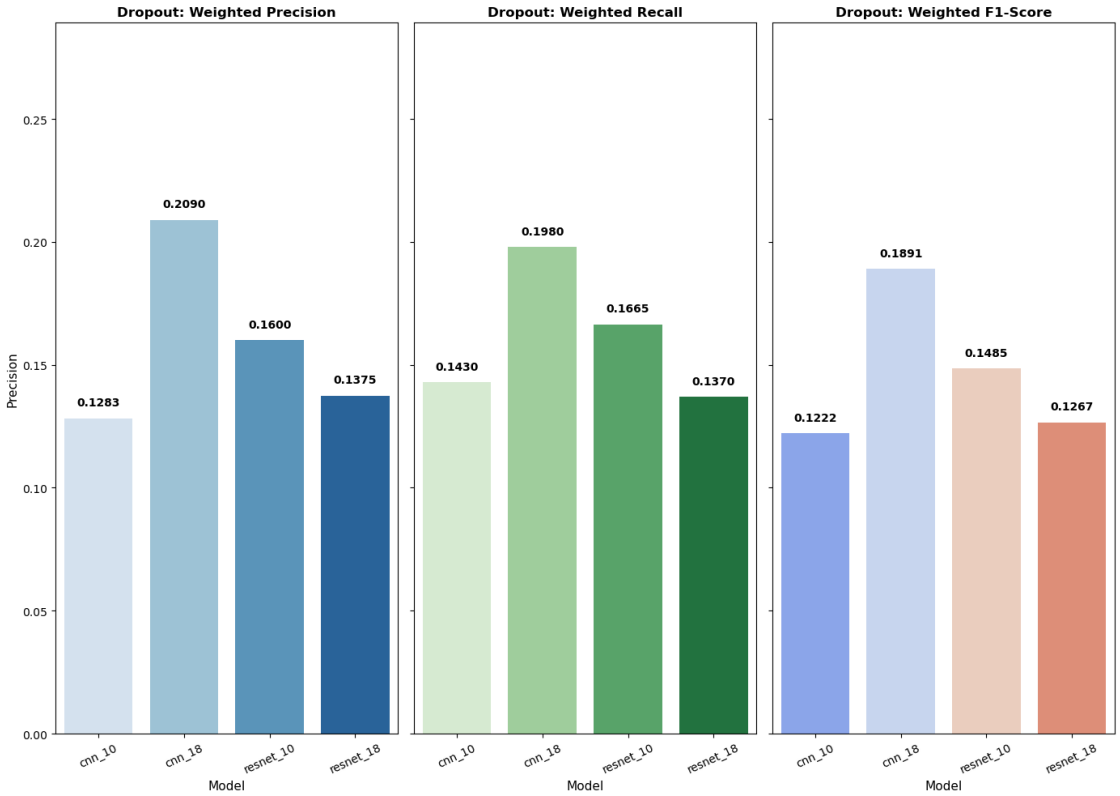
[33]:
```
plot_phase_results(dropout_histories, dropout_metrics, "Dropout")
```


Dropout: Training and Validation Loss
cnn_10 (Train)
cnn_10 (Val)
cnn_18 (Train)
cnn_18 (Val)
resnet_10 (Train)
resnet_10 (Val)
resnet_18 (Train)
resnet_18 (Val)
Loss
Epochs


Dropout: Top-1 Validation Accuracy
cnn_10
cnn_18
resnet_10
resnet_18
Top-1 Accuracy
Epochs



	Model Name	Phase	Weighted Precision	Weighted Recall	Weighted F1-Score
0	cnn_10	Dropout	0.128272	0.1430	0.122230
1	cnn_18	Dropout	0.208967	0.1980	0.189127
2	resnet_10	Dropout	0.159994	0.1665	0.148464
3	resnet_18	Dropout	0.137501	0.1370	0.126664



6.3.1 Evaluate Dropout Results

Dropout randomly zeroes out 50% of the neurons in the classification head during training. This forces the network to build redundant representations instead of relying on any single neuron, which directly targets overfitting.

CNN-18 benefited the most, reaching 19.8% top-1 accuracy which is the highest score of any custom model across all isolated technique experiments. Its F1-score (0.189) also leads. Dropout directly addresses the overfitting problem CNN-18 had at baseline where training loss dropped to near zero while validation loss kept climbing.

ResNet-10 improved from 14.65% to 16.65% top-1 accuracy. Even the shallow ResNet was overfitting somewhat at baseline, so regularization helped.

ResNet-18 didnt collapse like it did with data augmentation but still struggled at 13.7%. The deep architecture needs batch normalization for stable training; dropout alone cant fix gradient instability.

CNN-10 dropped from 17.1% to 14.3%. Dropout at 50% is too aggressive for a model with only 128 features going into the classifier. It doesnt have enough capacity to lose half its neurons and still perform.

Dropout works best on models that are overfitting (CNN-18, ResNet-10). It hurts models that are already underfitting (CNN-10) or models with fundamental gradient instability (ResNet-18).

6.4 CLIP Foundation Model

I implement CLIP (ViT-B/32) using linear probing where I freeze the pretrained vision backbone and train a single linear classification head on our labeled data. This gives CLIP access to the same training signal as the custom models while using features learned from 400 million image-text pairs during pretraining. The goal is to see how much pretrained feature quality matters compared to training from scratch.

[34]:
```
from transformers import CLIPModel, CLIPProcessor

class CLIPClassifier(nn.Module):
    """CLIP ViT-B/32 backbone with a trainable linear classification head."""
    def __init__(self, num_classes=20, freeze_backbone=True):
        super().__init__()
        print("Downloading/Loading CLIP backbone...")
        clip_model = CLIPModel.from_pretrained("openai/clip-vit-base-patch32")
        self.backbone = clip_model.vision_model

        if freeze_backbone:
            for param in self.backbone.parameters():
                param.requires_grad = False

        self.classifier = nn.Sequential(
            nn.Dropout(0.5),
            nn.Linear(self.backbone.config.hidden_size, num_classes)
        )

    def forward(self, x):
        outputs = self.backbone(pixel_values=x)
        x = outputs.pooler_output
        return self.classifier(x)
```

6.4.1 CLIP Data Preparation

[35]:
```
# CLIP has its own normalization stats, different from the ImageNet ones we use␣
↪for custom models
clip_transforms = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize((0.48145466, 0.4578275, 0.40821073), (0.26862954, 0.
↪26130258, 0.27577711))
])

full_dataset_clip = datasets.ImageFolder(data_dir, transform=clip_transforms)
```

```
clip_train_dataset = Subset(full_dataset_clip, train_dataset.indices)
clip_val_dataset = Subset(full_dataset_clip, val_dataset.indices)

clip_train_loader = DataLoader(clip_train_dataset, batch_size=BATCH_SIZE,␣
↪shuffle=True,
                               num_workers=NUM_WORKERS, pin_memory=True)
clip_val_loader = DataLoader(clip_val_dataset, batch_size=BATCH_SIZE,␣
↪shuffle=False,
                             num_workers=NUM_WORKERS, pin_memory=True)

print("CLIP DataLoaders created successfully.")
```

```
CLIP DataLoaders created successfully.
```

6.4.2 Train CLIP (Linear Probing)

[36]:
```
model_name = "clip"
weights_file = f"model_weights/clip/{model_name}.pth"

model = CLIPClassifier(num_classes=20, freeze_backbone=True).to(device)

history = load_weights(model, weights_file, device)
if history is not None:
    print(f"--- {model_name} loaded. Skipping training. ---")
else:
    print(f"--- Training {model_name} (Linear Probing) ---")
    criterion = nn.CrossEntropyLoss()
    optimizer = optim.Adam(model.classifier.parameters(), lr=0.001)
    history = train_model(model, clip_train_loader, clip_val_loader, criterion,␣
↪optimizer, epochs=21, device=device)
    save_weights(model, weights_file, history)
    print(f"--- Training {model_name} Completed ---")

clip_histories = {model_name: history}

# compute/load metrics
metrics_key = "clip"
precision, recall, f1, top1_acc, top5_acc, top_confused_pairs, y_true, y_pred =␣
↪\
    load_or_compute_metrics(metrics_key, model, clip_val_loader,␣
↪full_dataset_clip.classes, device)

if y_true is not None:
    plot_confusion_matrix(y_true, y_pred, class_names=full_dataset_clip.classes,
                          model_name="CLIP ViT-B/32 (Linear Probing)")
```

```
clip_metrics = [{
    "Model Name": model_name,
    "Weighted Precision": precision,
    "Weighted Recall": recall,
    "Weighted F1-Score": f1,
}]

print(f"{model_name} Processed | F1: {f1:.4f} | Top-1: {top1_acc:.4f} | Top-5:␣
↪{top5_acc:.4f}")
```

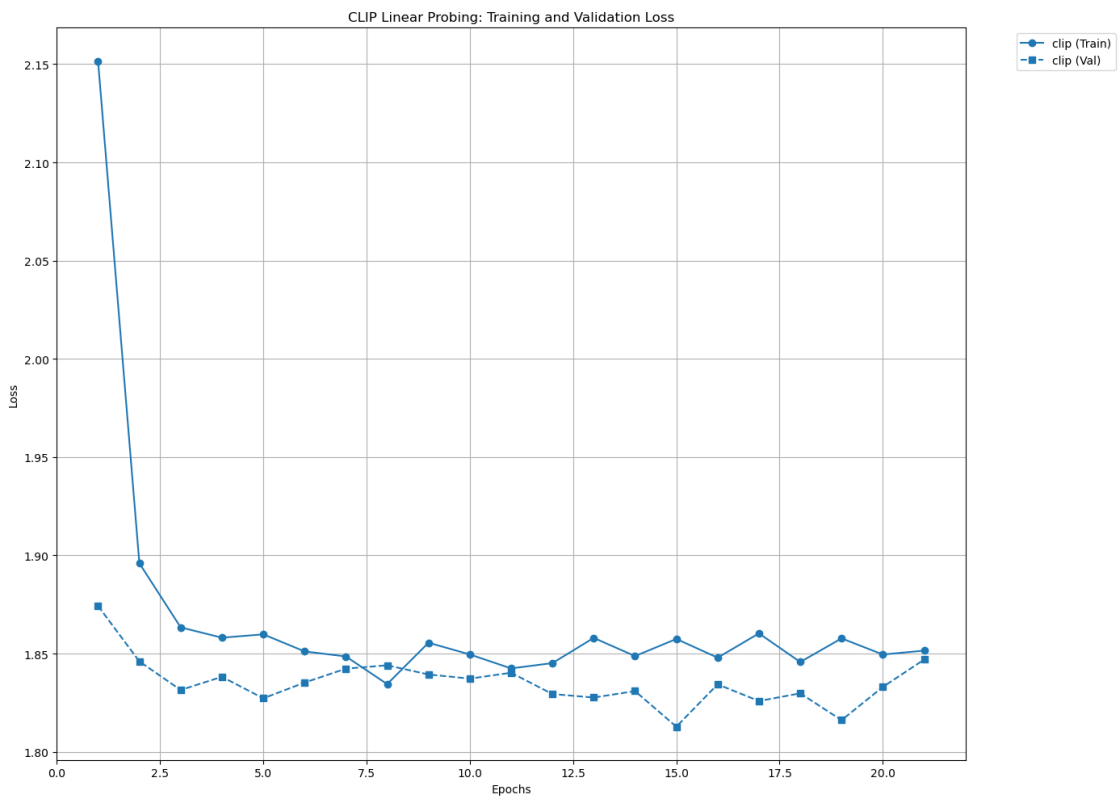
```
Downloading/Loading CLIP backbone…

Loading weights:   0%|          | 0/398 [00:00<?, ?it/s]

CLIPModel LOAD REPORT from: openai/clip-vit-base-patch32
Key                                  | Status     |  |
-------------------------------------+------------+--+-
text_model.embeddings.position_ids   | UNEXPECTED |  |
vision_model.embeddings.position_ids | UNEXPECTED |  |

Notes:
- UNEXPECTED        :can be ignored when loading from different
task/architecture; not ok if you expect identical arch.

Loaded weights from model_weights/clip/clip.pth
--- clip loaded. Skipping training. ---
Loaded existing metrics for clip
clip Processed | F1: 0.4352 | Top-1: 0.4445 | Top-5: 0.8220
```

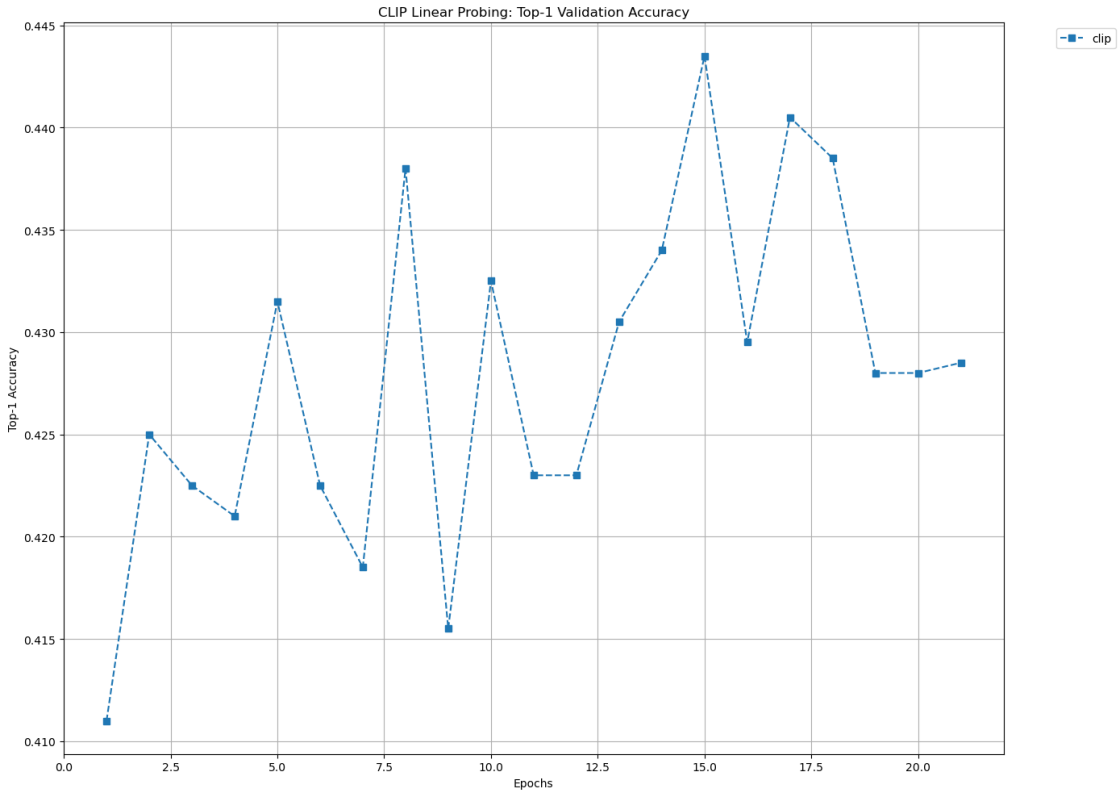
6.4.3 CLIP Linear Probing Results

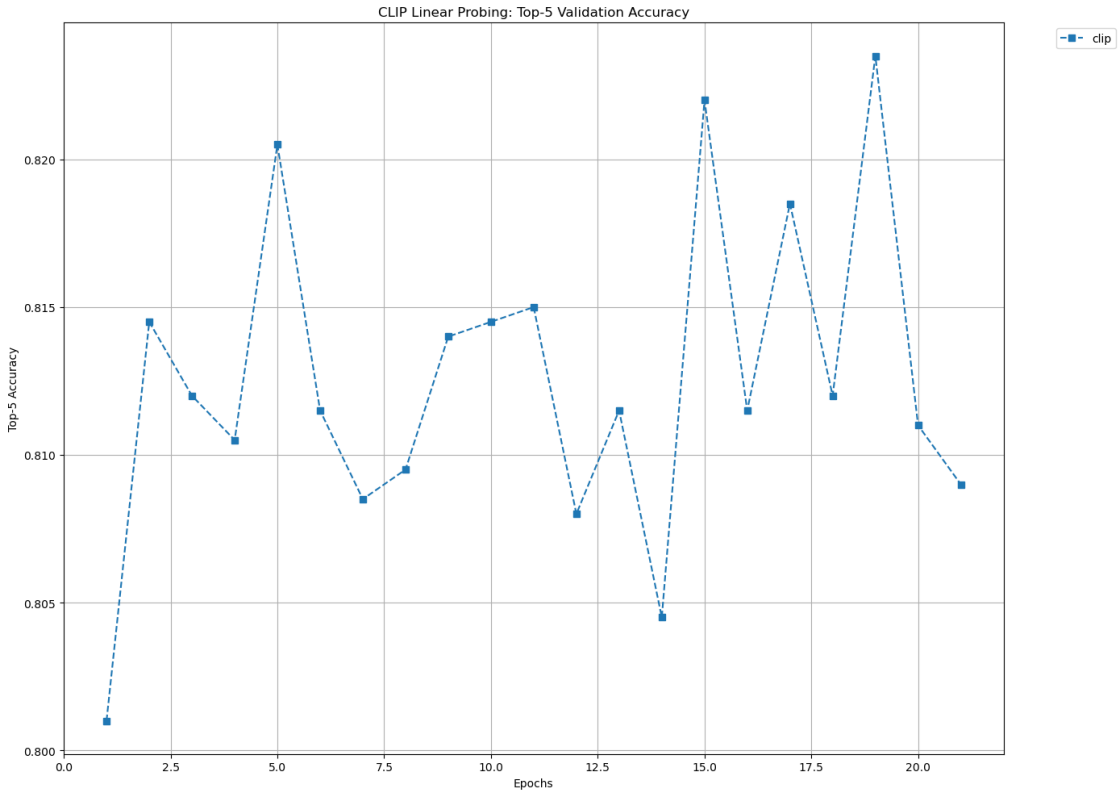
[37]:
```
print(f"{'='*60}")
print(f"FULL STATISTICAL REPORT: CLIP LINEAR PROBING")
print(f"{'='*60}\n")

clip_summary = pd.DataFrame({
    "Metric": ["Top-1 Accuracy", "Top-5 Accuracy", "Weighted Precision",␣
↪"Weighted Recall", "Weighted F1-Score"],
    "Score": [f"{top1_acc:.4f}", f"{top5_acc:.4f}", f"{precision:.4f}",␣
↪f"{recall:.4f}", f"{f1:.4f}"]
})
display(clip_summary)

print("\nTraining and Validation Curves:")
plot_loss_comparison(clip_histories, [model_name], title="CLIP Linear Probing:␣
↪Training and Validation Loss")
plot_accuracy_comparison(clip_histories, [model_name], acc_key='val_acc',␣
↪ylabel='Top-1 Accuracy', title="CLIP Linear Probing: Top-1 Validation␣
↪Accuracy")
```

```
plot_accuracy_comparison(clip_histories, [model_name], acc_key='val_top5_acc',␣
↪ylabel='Top-5 Accuracy', title="CLIP Linear Probing: Top-5 Validation␣
↪Accuracy")
```

```
============================================================
FULL STATISTICAL REPORT: CLIP LINEAR PROBING
============================================================


               Metric   Score
0      Top-1 Accuracy  0.4445
1      Top-5 Accuracy  0.8220
2  Weighted Precision  0.4393
3     Weighted Recall  0.4445
4   Weighted F1-Score  0.4352


Training and Validation Curves:
```




CLIP Linear Probing: Top-1 Validation Accuracy
Top-1 Accuracy
Epochs
clip
0.445
0.440
0.435
0.430
0.425
0.420
0.415
0.410
0.0
2.5
5.0
7.5
10.0
12.5
15.0
17.5
20.0


CLIP Linear Probing: Top-5 Validation Accuracy
Top-5 Accuracy
Epochs
clip
0.820
0.815
0.810
0.805
0.800
0.0
2.5
5.0
7.5
10.0
12.5
15.0
17.5
20.0

6.4.4 Evaluate Foundation Model Results

CLIP linear probing gets 44.45% top-1 and 82.2% top-5 accuracy which is more than double the best custom model. A detailed comparison and analysis is in Section 9.

7 (7.) Performance Evaluation

I use the following metrics to evaluate all models:

Top-1 Accuracy measures how often the model's single best prediction matches the true genre. This is the most intuitive metric but can be harsh on a 20-class problem where random chance is 5%.

Top-5 Accuracy measures how often the true genre appears in the model's top 5 predictions. This is useful because many genres are visually similar (metal subgenres, country/folk/blues) so a model that ranks the correct genre in its top 5 is still capturing meaningful signal even if it doesnt get the exact prediction right.

Weighted Precision, Recall, and F1-Score are computed using sklearn with weighted averaging. Weighted averaging accounts for class balance by weighting each genre's score by its support count. Since this dataset is balanced (1,000 per genre), weighted and macro averages are similar, but weighted is the safer default.

Confusion Matrices show the full prediction distribution across all 20 genres. These reveal which specific genre pairs the models confuse most, which is more informative than a single accuracy number.

These metrics are appropriate for this task because album art classification is a multi-class problem with visually similar classes. Top-1 alone would make every model look bad since 20% is a reasonable ceiling for custom models on this data. Top-5, F1, and confusion analysis give a more complete picture of what the models are actually learning.

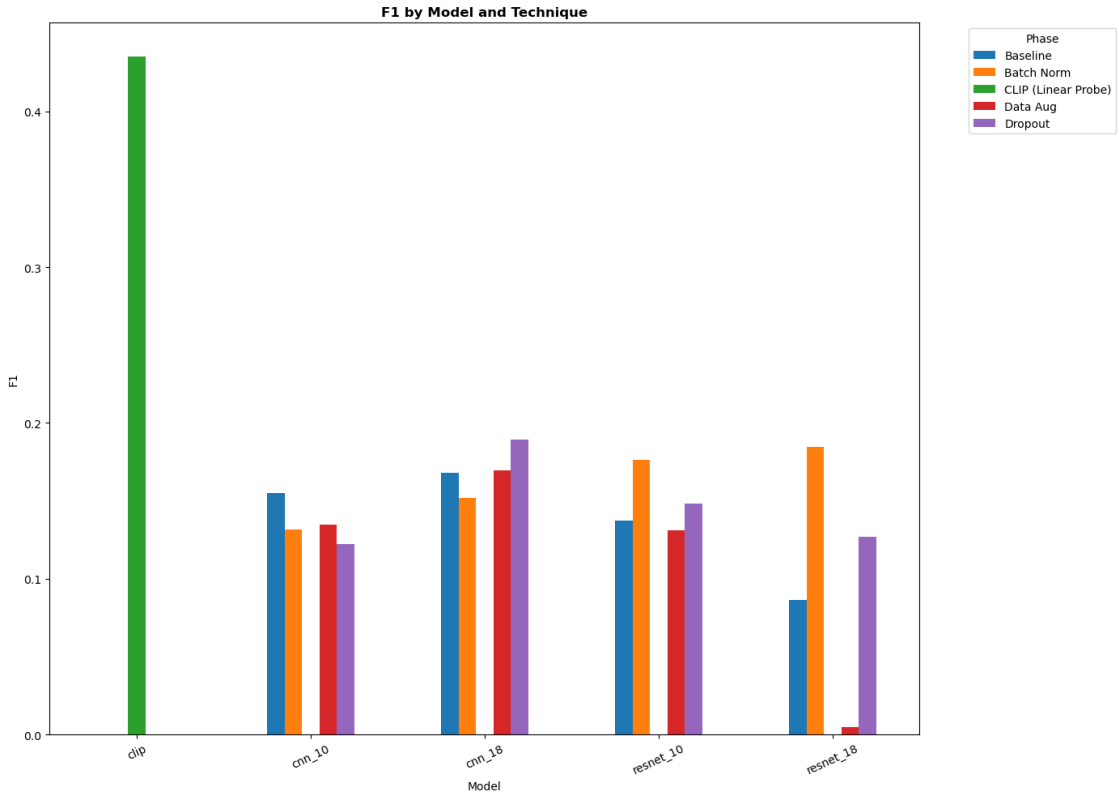
[38]:
```
all_metrics = metrics_store.load_all()

# collect results for each phase
phases = [
    ("Baseline", "baseline"),
    ("Batch Norm", "batchnorm"),
    ("Data Aug", "data_aug"),
    ("Dropout", "dropout"),
]

comparison_rows = []
for phase_label, prefix in phases:
    for _, mname in core_models:
        key = f"{prefix}_{mname}"
        m = all_metrics.get(key, {})
```

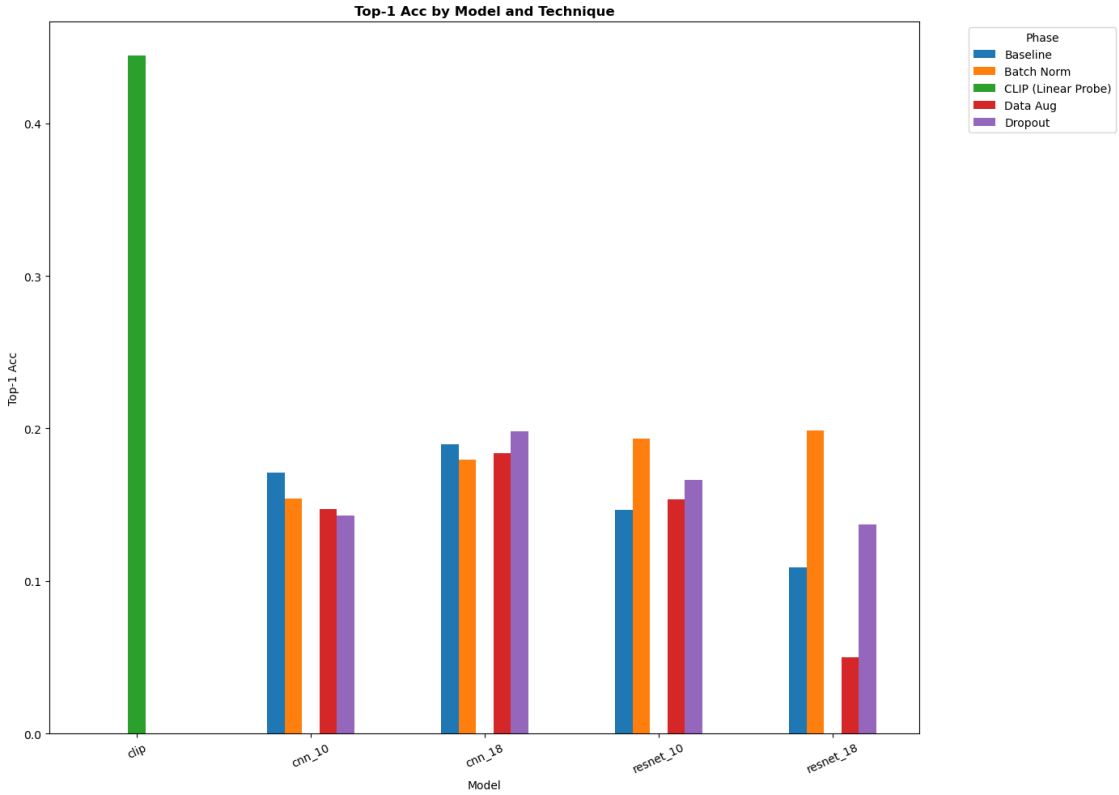
```
        if m:
            comparison_rows.append({
                "Phase": phase_label,
                "Model": mname,
                "Precision": m.get("precision", 0),
                "Recall": m.get("recall", 0),
                "F1": m.get("f1", 0),
                "Top-1 Acc": m.get("top1_acc", 0),
                "Top-5 Acc": m.get("top5_acc", 0),
            })

# add CLIP linear probing
clip_m = all_metrics.get("clip", {})
if clip_m:
    comparison_rows.append({
        "Phase": "CLIP (Linear Probe)",
        "Model": "clip",
        "Precision": clip_m.get("precision", 0),
        "Recall": clip_m.get("recall", 0),
        "F1": clip_m.get("f1", 0),
        "Top-1 Acc": clip_m.get("top1_acc", 0),
        "Top-5 Acc": clip_m.get("top5_acc", 0),
    })

comparison_df = pd.DataFrame(comparison_rows)
display(comparison_df)
```

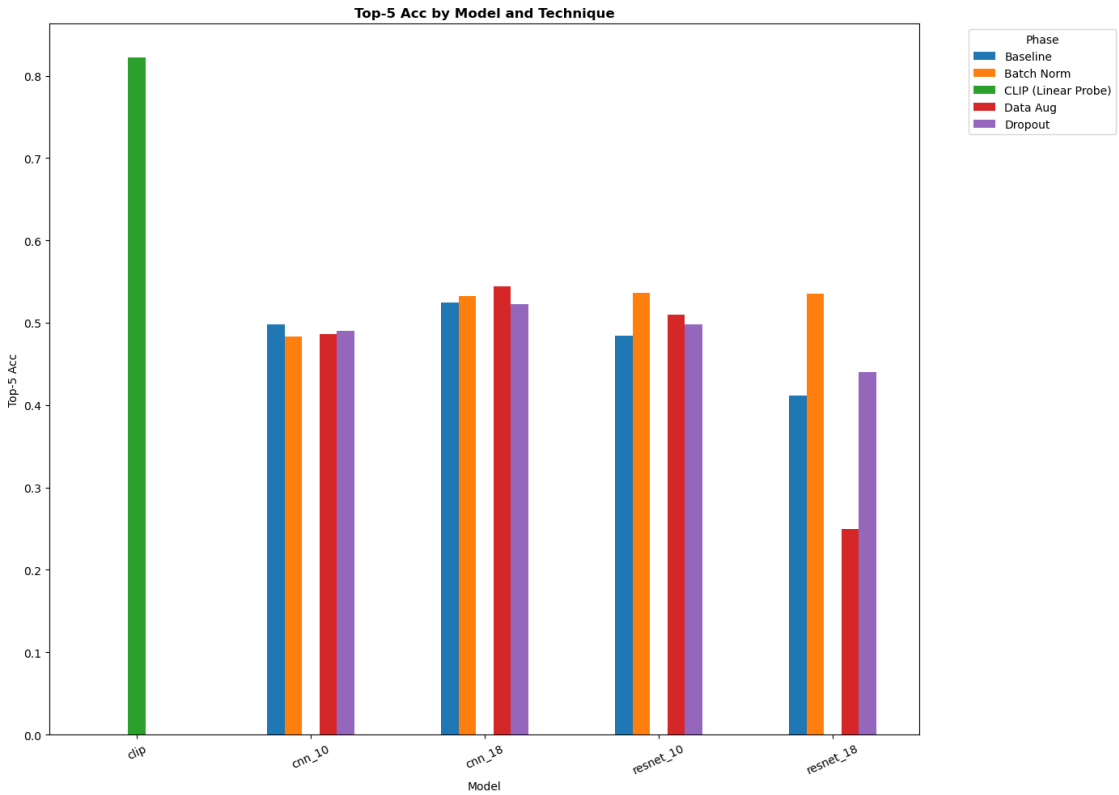
```
                  Phase      Model  Precision  Recall        F1  Top-1 Acc  \
0              Baseline     cnn_10   0.167924  0.1710  0.154777     0.1710
1              Baseline     cnn_18   0.199849  0.1895  0.168056     0.1895
2              Baseline  resnet_10   0.144964  0.1465  0.137130     0.1465
3              Baseline  resnet_18   0.103118  0.1090  0.086242     0.1090
4            Batch Norm     cnn_10   0.139439  0.1540  0.131537     0.1540
5            Batch Norm     cnn_18   0.167364  0.1795  0.152006     0.1795
6            Batch Norm  resnet_10   0.205456  0.1935  0.176521     0.1935
7            Batch Norm  resnet_18   0.218453  0.1985  0.184498     0.1985
8              Data Aug     cnn_10   0.147706  0.1470  0.134898     0.1470
9              Data Aug     cnn_18   0.187527  0.1840  0.169670     0.1840
10             Data Aug  resnet_10   0.140506  0.1535  0.131331     0.1535
11             Data Aug  resnet_18   0.002500  0.0500  0.004762     0.0500
12              Dropout     cnn_10   0.128272  0.1430  0.122230     0.1430
13              Dropout     cnn_18   0.208967  0.1980  0.189127     0.1980
14              Dropout  resnet_10   0.159994  0.1665  0.148464     0.1665
15              Dropout  resnet_18   0.137501  0.1370  0.126664     0.1370
16  CLIP (Linear Probe)       clip   0.439269  0.4445  0.435224     0.4445

    Top-5 Acc
0      0.4985
```

```
1       0.5245
2       0.4845
3       0.4120
4       0.4830
5       0.5325
6       0.5360
7       0.5355
8       0.4860
9       0.5440
10      0.5095
11      0.2500
12      0.4905
13      0.5230
14      0.4985
15      0.4405
16      0.8220
```

[39]:
```
# individual bar charts for each metric
if not comparison_df.empty:
    for metric in ["F1", "Top-1 Acc", "Top-5 Acc"]:
        fig, ax = plt.subplots(figsize=plot_size)
        pivot = comparison_df.pivot(index="Model", columns="Phase",␣
 ↪values=metric)
        pivot.plot(kind="bar", ax=ax, rot=25)
        ax.set_title(f"{metric} by Model and Technique", fontsize=12,␣
 ↪fontweight="bold")
        ax.set_ylabel(metric)
        ax.legend(title="Phase", bbox_to_anchor=(1.05, 1), loc='upper left')
        plt.tight_layout()
        plt.show()
```


F1 by Model and Technique
Phase
Baseline
Batch Norm
CLIP (Linear Probe)
Data Aug
Dropout
F1
0.0
0.1
0.2
0.3
0.4
clip
cnn_10
cnn_18
resnet_10
resnet_18
Model


Top-1 Acc by Model and Technique
Phase
Baseline
Batch Norm
CLIP (Linear Probe)
Data Aug
Dropout
Top-1 Acc
0.0
0.1
0.2
0.3
0.4
clip
cnn_10
cnn_18
resnet_10
resnet_18
Model



8 (8.) Comparison of Approaches

Batch normalization is the best individual technique. It gave the most consistent improvement across the deep models. ResNet-18 went from worst performer (10.9%) to competitive (19.85%) with batch norm; this is the biggest single improvement from any technique. The deeper the model, the more it benefits from having stable activations.

Dropout was the best technique for CNN-18 specifically. CNN-18 achieved its top score in top-1 accuracy (19.8%) with dropout, which outperformed its batch-norm score (17.95%). This makes sense because dropout directly addresses CNN-18s overfitting problem where training loss was near zero while validation loss kept going up.

Data augmentation was the riskiest technique. It helped CNN-18 a bit on top-5 accuracy but caused ResNet-18 to collapse entirely. Without batch norm to stabilize the deep network, the varied augmented inputs caused mode collapse. You can not just apply data augmentation blindly; it interacts with model depth and whether or not you have normalization.

The shallow models (CNN-10, ResNet-10) rarely benefit from any technique. They dont have enough depth to overfit badly so regularization solves a problem they dont have. Normalization adds overhead without enough depth to benefit from it.

CLIP outperformed everything by a factor of 2 which is analyzed in detail in Section 9.

How do the results compare to my hypotheses from the proposal? - Hypothesis 1 (ResNet > CNN): Partially confirmed. With batch normalization, ResNet-18 is the top custom model. Without BN, the CNNs actually win because the ResNets suffer more from gradient instability. - Hypothesis 2 (depth helps ResNet, hurts CNN): Confirmed for ResNet with BN. At baseline both deep models overfit badly, but ResNet-18 recovers with normalization while CNN-18 benefits more from dropout. - Hypothesis 3 (CLIP > all): Confirmed. CLIP dominates by 2x+ on every metric.

9 (9.) Foundation Model Comparison

I implemented CLIP (ViT-B/32) using linear probing where the pretrained vision backbone is frozen and a single linear classification head is trained on our labeled data. This gave CLIP access to the same training signal as the custom models while leveraging features learned from 400 million image-text pairs.

CLIP linear probing gets 44.45% top-1 and 82.2% top-5 accuracy, more than doubling the best custom model (CNN-18 dropout at 19.8%). The gap between CLIP and the custom models is entirely about feature quality. CLIP already knows what edges, textures, and compositions look like from its massive pretraining dataset. Our custom models have to learn all of that from scratch with only 16k training images which is not enough to reach that level of feature quality.

The proposal originally described CLIP as zero-shot only. I changed to linear probing during implementation because it provides a fairer comparison; CLIP gets the same labeled data as the custom models and a linear classification head is trained which is more similar to how the other models were trained.

---

10 (10.) Error and Failure Analysis

10.0.1 Most Confused Class Pairs

Which genre pairs were confused the most, and by which models?

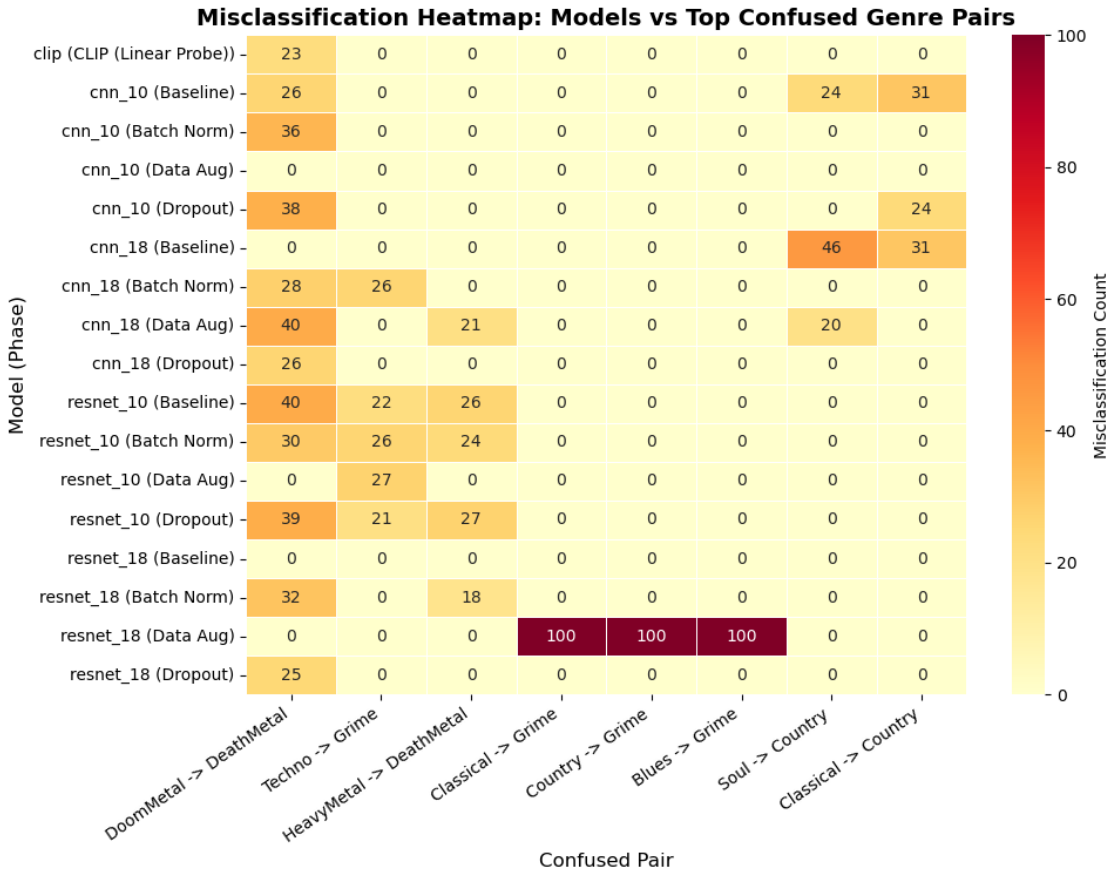
[40]:
```
# collect confused pairs from all models (debug excluded)
confused_rows = []
all_phases = phases + [("CLIP (Linear Probe)", "clip")]

for phase_label, prefix in all_phases:
    if prefix == "clip":
        keys = [(prefix, prefix)]
    else:
        keys = [(f"{prefix}_{mname}", mname) for _, mname in core_models]

    for metrics_key, display_name in keys:
        m = all_metrics.get(metrics_key, {})
```

```
        pairs = m.get("top_confused_pairs", [])
        for pair in pairs:
            confused_rows.append({
                "Phase": phase_label,
                "Model": display_name,
                "Pair": f"{pair['true']} -> {pair['predicted']}",
                "Count": pair["count"]
            })
```

10.0.2 Confusion Heatmap: Models vs Confused Pairs

Which models struggle most with which genre pairs?

[41]:
```
confused_df = pd.DataFrame(confused_rows)

if not confused_df.empty:
    # build a label for each model+phase combination
    confused_df["Model Label"] = confused_df["Model"] + " (" +␣
↪confused_df["Phase"] + ")"

    # pivot into a matrix: rows = model labels, columns = confused pairs
    heatmap_data = confused_df.pivot_table(index="Model Label", columns="Pair",
                                           values="Count", aggfunc="sum",␣
↪fill_value=0)

    # keep only top 8 most-confused pairs for readability
    HEATMAP_TOP_N = 8
    col_order = heatmap_data.sum().sort_values(ascending=False).index[:
↪HEATMAP_TOP_N]
    heatmap_data = heatmap_data[col_order]

    plt.figure(figsize=(max(10, len(col_order) * 1.2), max(6, len(heatmap_data)␣
↪* 0.45)))
    sns.heatmap(heatmap_data, annot=True, fmt="d", cmap="YlOrRd", linewidths=0.
↪5,
                cbar_kws={"label": "Misclassification Count"})
    plt.title("Misclassification Heatmap: Models vs Top Confused Genre Pairs",␣
↪fontsize=14, fontweight="bold")
    plt.xlabel("Confused Pair", fontsize=12)
    plt.ylabel("Model (Phase)", fontsize=12)
    plt.xticks(rotation=35, ha='right', fontsize=10)
    plt.yticks(fontsize=10)
    plt.tight_layout()
    plt.show()
```



10.0.3 Patterns in Misclassifications

The most confused pair across nearly all models is DoomMetal → DeathMetal. Both genres share dark color palettes, aggressive typography, and similar artwork styles like skulls and dark landscapes. The models consistently cant separate visually similar subgenres. HeavyMetal and DeathMetal confusions follow the same pattern.

Country, Folk, Soul, and Blues form a confusion cluster. At baseline many models predict "Country" for Soul, Blues, Folk, and Classical inputs. These genres share warm tones, natural landscapes, and portrait-style photographs. There isnt enough visual difference between these covers for a CNN to reliably separate them.

Techno and Grime get confused frequently, especially by the ResNet models. Both are electronic music genres and their album art tends toward neon colors and abstract digital aesthetics.

CLIP reduces these confusions but doesnt eliminate them. CLIP still confuses DoomMetal → DeathMetal (23 counts) and Electronic → Pop (22 counts) but at much lower rates than the custom models. The confusions CLIP makes are at the boundary of genuine visual similarity rather than wholesale misclassification.

10.0.4 Limitations

There is a ceiling on visual-only genre classification. Album art is designed by artists and designers; it is not algorithmically mapped to genre. A rock album can look like a jazz cover and a hip-hop album can look like an electronic one. The visual signal for genre is weak and inconsistent which caps accuracy for any vision-only approach.

The dataset has 1,000 images per genre which is relatively small for a 20-class problem.

I deliberately isolated the visual channel to study what a CNN can extract from album art alone. Real genre classification would benefit from audio features, artist metadata, and lyrics. The low accuracy ceiling (~20% for custom models) reflects that the cover image discards most of the information that actually defines genre.

Training deep networks from scratch on 16,000 images is not going to produce great results on a 20-class problem. CLIP getting 44% suggests the bottleneck is feature quality not model architecture; pretrained features break through the ceiling that training from scratch cant.

11 (11.) Team Collaboration

I completed this project individually. I used JupyterNotebook as the primary development environment which allowed me to keep code, analysis, and writeups organized in a single document. Model weights and metrics are saved to disk so I could train models across multiple sessions without losing progress.

12 (12.) Individual Contributions

Since this was a solo project, I was responsible for all aspects: - Dataset selection, download pipeline, and preprocessing - Design and implementation of all 4 CNN/ResNet architectures - Training pipeline with early stopping, weight saving/loading, and metric caching - Implementation of all 3 performance improvement techniques (batch normalization, data augmentation, dropout) each in isolation - CLIP foundation model integration (linear probing with frozen backbone) - All evaluation code including confusion matrices, F-score plots, and comparison tables - Written analysis for every section of this report

---

13 (13.) Conclusion

I built 4 custom CNN/ResNet architectures and compared them against CLIP to classify song genre from album cover art across 20 genres.

Training from scratch on 20k images gets about 15-20% top-1 accuracy. Album cover art has a weak but learnable visual signal for genre; the models are picking up real patterns, not random noise.

Batch normalization is the most impactful technique for deep architectures. It nearly doubled ResNet-18's accuracy by stabilizing gradient flow through the residual blocks. Without it, the deep ResNet was the worst performer; with it, the best custom model.

I learned that techniques need to be matched to the architecture. Dropout helps models that are overfitting (CNN-18) but hurts models that are underfitting (CNN-10). Data augmentation may cause mode collapse on deep networks without batch normalization. Blindly applying techniques without understanding the model's failure mode can make things worse. The correct model architecture and performance improvement technique combination is what is necessary to properly approach a problem like this.

CLIP linear probing gets 44.45% top-1 and 82.2% top-5 accuracy, outperforming all custom models by more than 2x. Pretrained features from 400 million image-text pairs are significantly better than anything we can learn from scratch on this dataset size.

Genre confusion follows visual similarity. Metal subgenres cluster together, Country/Folk/Soul/Blues cluster together. The models are learning real visual associations between album art and genre even when overall accuracy is low.

13.0.1 What I Would Do Differently

- I tested each technique in isolation which gives cleaner insights, but I didnt get to test the optimal combination of performance improvedment techniques which would lead to the best custom model performance.
- Reproducibility (seeds, deterministic flags, hardware documentation) should have been set up from the start instead of added after training.

13.0.2 Future Work

- Fine-tune CLIP's backbone by unfreezing some layers to see if the accuracy gap narrows further.
- Merge visually indistinguishable subgenres (like DoomMetal/DeathMetal) into parent genres to reduce the confusion clusters.

[ ]:

```

```

[ ]:

```

```

[ ]:

```

```

[ ]:

```

```